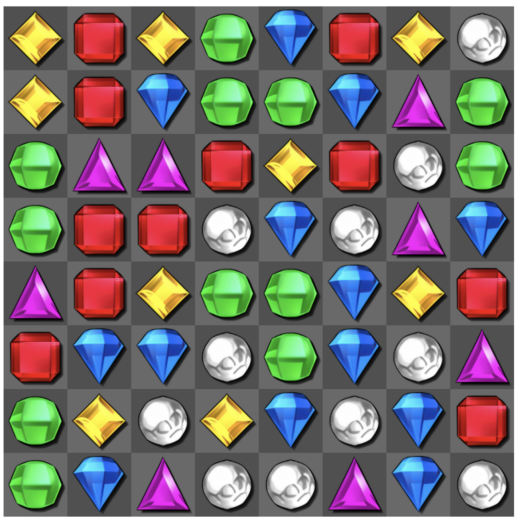
Projet Gemmes et Algorithmes



Généré par Doxygen 1.15.0

1 Index des structures de données **1**

1.1 Structures de données 1

2 Index des fichiers **3**

2.1 Liste des fichiers 3

3 Documentation des structures de données **5**

3.1 Référence de la structure board 5

3.1.1 Description détaillée 5

3.1.2 Documentation des champs 6

3.1.2.1 dummy 6

3.2 Référence de la structure cell 6

3.2.1 Description détaillée 6

3.2.2 Documentation des champs 7

3.2.2.1 object 7

3.2.2.2 power 7

3.2.2.3 countdown 7

3.2.2.4 [union] 7

3.3 Référence de la structure game 7

3.3.1 Description détaillée 8

3.3.2 Documentation des champs 9

3.3.2.1 b 9

3.3.2.2 puzzle 9

3.3.2.3 lines 9

3.3.2.4 upgrades 10

3.3.2.5 gravity_control 10

3.3.2.6 nodelay 10

3.3.2.7 easy 10

3.3.2.8 gravity_gems 10

3.3.2.9 bombs 11

3.3.2.10 refill_mode 11

3.3.2.11 score 11

3.4 Référence de la structure hash_table 11

3.4.1 Description détaillée 12

3.4.2 Documentation des champs 13

3.4.2.1 hash_capacity 13

3.4.2.2 hash_size 13

3.4.2.3 hash_table 13

3.4.2.4 element_size . . . 13
3.5 Référence de la structure score_block . . . 13
3.5.1 Description détaillée . . . 14
3.5.2 Documentation des champs . . . 14
3.5.2.1 level . . . 14
3.5.2.2 level_score . . . 14
3.5.2.3 target_score . . . 15
3.5.2.4 scale_score . . . 15
3.5.2.5 scores . . . 15
3.5.2.6 refills . . . 15
3.6 Référence de la structure score_table . . . 15
3.6.1 Description détaillée . . . 16
3.6.2 Documentation des champs . . . 16
3.6.2.1 names . . . 16
3.6.2.2 scores . . . 17
3.6.2.3 size . . . 17
3.6.2.4 player_index . . . 17
3.6.2.5 permutation . . . 17
3.7 Référence de la structure solver_move . . . 17
3.7.1 Description détaillée . . . 18
3.7.2 Documentation des champs . . . 18
3.7.2.1 type . . . 18
3.7.2.2 index . . . 18
3.7.2.3 dir . . . 19
3.7.2.4 next . . . 19

4 Documentation des fichiers **21**

4.1 Référence du fichier alloc.h . . . 21
4.1.1 Description détaillée . . . 22
4.1.2 Documentation des macros . . . 22
4.1.2.1 CREATE_MEMORY . . . 22
4.1.2.2 CREATE_ZERO_MEMORY . . . 23
4.1.2.3 REDIMENSION_MEMORY . . . 24
4.1.2.4 CHECK_NULL . . . 25
4.1.3 Documentation des fonctions . . . 26
4.1.3.1 check_null() . . . 26
4.1.3.2 multiple_free() . . . 26
4.2 alloc.h . . . 27

4.3 Référence du fichier error.h . . . 28
4.3.1 Description détaillée . . . 29
4.3.2 Documentation des macros . . . 30
4.3.2.1 ATTENTION . . . 30
4.3.2.2 HINT . . . 30
4.3.2.3 OK . . . 31
4.3.2.4 KO . . . 31
4.3.2.5 DEBUG_LEVEL . . . 31
4.3.2.6 PRINT_ERROR . . . 32
4.3.2.7 FATAL . . . 32
4.3.2.8 PRINT_DEBUG . . . 33
4.3.2.9 TRACE . . . 33
4.3.2.10 DEBUG . . . 34
4.3.2.11 INFO . . . 34
4.3.2.12 WARNING . . . 34
4.3.2.13 ERROR . . . 35
4.3.2.14 CRITICAL . . . 35
4.3.3 Documentation du type de l'énumération . . . 35
4.3.3.1 debug_level . . . 35
4.3.3.2 msg_level . . . 35
4.4 error.h . . . 36
4.5 Référence du fichier game.h . . . 37
4.5.1 Description détaillée . . . 40
4.5.2 Documentation du type de l'énumération . . . 40
4.5.2.1 direction . . . 40
4.5.2.2 element . . . 40
4.5.2.3 gem_power . . . 41
4.5.2.4 score_type . . . 42
4.5.3 Documentation des fonctions . . . 42
4.5.3.1 board_get_width() . . . 42
4.5.3.2 board_get_height() . . . 43
4.5.3.3 board_get_gravity() . . . 43
4.5.3.4 board_get_picked() . . . 43
4.5.3.5 board_get_cell() . . . 44
4.5.3.6 board_refill_size() . . . 44
4.5.3.7 board_read_refill() . . . 45
4.5.3.8 game_create_classic() . . . 45

4.5.3.9 game_create_puzzle() . . . 46
4.5.3.10 game_next_level() . . . 48
4.5.3.11 game_release() . . . 48
4.5.3.12 game_clone() . . . 49
4.6 game.h . . . 49
4.7 Référence du fichier game_ias.h . . . 53
4.7.1 Description détaillée . . . 53
4.7.2 Documentation du type de l'énumération . . . 54
4.7.2.1 ia_classic . . . 54
4.7.3 Documentation des fonctions . . . 54
4.7.3.1 ia_random() . . . 54
4.7.3.2 ia_safe() . . . 55
4.7.3.3 ia_agressive() . . . 55
4.7.4 Documentation des variables . . . 56
4.7.4.1 ia_functions . . . 56
4.7.4.2 ia_names . . . 56
4.8 game_ias.h . . . 56
4.9 Référence du fichier game_play.h . . . 57
4.9.1 Description détaillée . . . 59
4.9.2 Documentation du type de l'énumération . . . 59
4.9.2.1 anim_type . . . 59
4.9.2.2 end_signal . . . 60
4.9.2.3 gem_kill . . . 60
4.9.3 Documentation des fonctions . . . 61
4.9.3.1 game_init_animation() . . . 61
4.9.3.2 game_classic_score() . . . 62
4.9.3.3 game_classic_end() . . . 62
4.9.3.4 game_puzzle_end() . . . 62
4.9.3.5 game_handle_mouse_click() . . . 63
4.9.3.6 game_handle_hint_request() . . . 63
4.9.3.7 game_handle_detonation() . . . 64
4.9.3.8 game_set_gravity() . . . 64
4.9.3.9 game_cycle_gravity() . . . 64
4.9.3.10 game_handle_refill() . . . 65
4.9.3.11 game_end_animation() . . . 65
4.9.3.12 game_get_cell_move() . . . 66
4.9.3.13 game_get_cell_kill() . . . 67

4.9.3.14 game_retrieve_hint() . . . 67
4.9.4 Documentation des variables . . . 68
4.9.4.1 game_anim_signal . . . 68
4.9.4.2 game_end_signal . . . 68
4.10 game_play.h . . . 68
4.11 Référence du fichier game_save.h . . . 71
4.11.1 Description détaillée . . . 72
4.11.2 Documentation du type de l'énumération . . . 72
4.11.2.1 save_keywords . . . 72
4.11.3 Documentation des fonctions . . . 73
4.11.3.1 save_game() . . . 73
4.11.3.2 load_game() . . . 73
4.11.4 Documentation des variables . . . 74
4.11.4.1 save_keyword_strings . . . 74
4.12 game_save.h . . . 74
4.13 Référence du fichier game_scores.h . . . 75
4.13.1 Description détaillée . . . 76
4.13.2 Documentation des fonctions . . . 76
4.13.2.1 load_score_table() . . . 76
4.13.2.2 save_score_table() . . . 76
4.13.2.3 delete_score_table() . . . 77
4.13.2.4 sort_score_table() . . . 77
4.14 game_scores.h . . . 77
4.15 Référence du fichier game_solver.h . . . 78
4.15.1 Documentation des définitions de type . . . 80
4.15.1.1 solver_move . . . 80
4.15.2 Documentation du type de l'énumération . . . 80
4.15.2.1 solver_move_type . . . 80
4.15.2.2 solver_result_type . . . 80
4.15.2.3 solver . . . 81
4.15.3 Documentation des fonctions . . . 81
4.15.3.1 create_hash_table() . . . 81
4.15.3.2 delete_hash_table() . . . 81
4.15.3.3 hash_table_insert() . . . 82
4.15.3.4 solve_puzzle() . . . 82
4.15.3.5 game_implement_solver_move() . . . 83
4.16 game_solver.h . . . 83

4.17 Référence du fichier project_parameters.h . . . 85
4.17.1 Description détaillée . . . 87
4.17.2 Documentation des macros . . . 87
4.17.2.1 WINDOW_SCREENWIDTH . . . 87
4.17.2.2 WINDOW_SCREENHEIGHT . . . 88
4.17.2.3 GAME_WIDTH . . . 88
4.17.2.4 GAME_HEIGHT . . . 88
4.17.2.5 GAME_LINES . . . 88
4.17.2.6 GAME_NODELAY . . . 88
4.17.2.7 GAME_EASY . . . 89
4.17.2.8 GAME_WRAP . . . 89
4.17.2.9 GAME_BOMBS . . . 89
4.17.2.10 GAME_UPGRADES . . . 89
4.17.2.11 GAME_REFILL_MODE . . . 89
4.17.2.12 GAME_BALANCED . . . 90
4.17.2.13 GAME_STORAGE_INITIAL . . . 90
4.17.2.14 GAME_CLEARSCORE . . . 90
4.17.2.15 GAME_CLEARMULTI . . . 90
4.17.2.16 TEST_WIDTH . . . 90
4.17.2.17 TEST_HEIGHT . . . 91
4.17.2.18 TEST_PUZZLE . . . 91
4.17.2.19 TEST_LINES . . . 91
4.17.2.20 TEST_NODELAY . . . 91
4.17.2.21 TEST_EASY . . . 91
4.17.2.22 TEST_WRAP . . . 92
4.17.2.23 TEST_BOMBS . . . 92
4.17.2.24 TEST_UPGRADES . . . 92
4.17.2.25 TEST_REFILL_MODE . . . 92
4.17.2.26 TEST_STORAGE_INITIAL . . . 92
4.17.2.27 TEST_BALANCED . . . 93
4.17.2.28 SOLVER_MAX_CONFIGS . . . 93
4.18 project_parameters.h . . . 93
4.19 Référence du fichier project_tools.h . . . 95
4.19.1 Description détaillée . . . 95
4.19.2 Documentation des macros . . . 96
4.19.2.1 MAX . . . 96
4.19.2.2 MIN . . . 96

4.19.2.3 SWAP 96
4.19.3 Documentation des définitions de type 97
4.19.3.1 uchar 97
4.19.3.2 ushort 97
4.19.3.3 uint 97
4.19.3.4 ulong 97
4.20 project_tools.h 97

Index **99**

Chapitre 1

Index des structures de données

1.1 Structures de données

Liste des structures de données avec une brève description :

board
Structure représentant un plateau de jeu 5
cell
Structure représentant une case. Utilisée pour communiquer le contenu d'une case à l'interface graphique 6
game
Structure représentant un jeu complet 7
hash_table
Solveur pour le mode puzzle 11
score_block
Structure pour contenir les scores d'un jeu 13
score_table
Structure permettant de stocker le tableau des scores affiché par l'interface graphique (GUI) 15
solver_move
Structure récursive représentant une séquence de coups d'une solution de puzzle 17

Chapitre 2

Index des fichiers

2.1 Liste des fichiers

Liste de tous les fichiers avec une brève description :

alloc.h
: Macros et fonctions d'allocation mémoire 21

error.h
: Macros permettant d'afficher des messages d'erreur 28

game.h
: Implémentation du jeu 37

game_ias.h
: IA pour le mode classique 53

game_play.h
: Fonctions appelées par l'interface (GUI) pour gérer les entrées utilisateur. C'est ici que la logique du jeu doit être implémentée 57

game_save.h
: Sauvegarde et chargement des états de jeu 71

game_scores.h
: Gestion des meilleurs scores en mode classique. Contient les fonctions permettant de charger, sauvegarder et trier les meilleurs scores à partir du fichier de scores . 75

game_solver.h 78

project_parameters.h
: Paramètres 85

project_tools.h
: Tools 95

Chapitre 3

Documentation des structures de données

3.1 Référence de la structure board

Structure représentant un plateau de jeu.

`#include <game.h>`

Champs de données

— int dummy

3.1.1 Description détaillée

Structure représentant un plateau de jeu.

Cette structure est utilisée pour représenter un plateau de jeu. L'implémentation réelle du plateau vous appartient, mais elle doit au moins contenir les informations suivantes :

- la largeur et la hauteur du plateau,
- la direction de la gravité,
- la case actuellement sélectionnée par le joueur (ou un indicateur indiquant qu'aucune case n'est sélectionnée),
- le contenu de chaque case, et
- la file de stockage si elle est utilisée (uniquement en mode puzzle dans un mode nécessitant une file de stockage).

Vous pouvez ajouter des champs supplémentaires si nécessaire pour votre implémentation (cela sera utile pour stocker des informations lorsqu'une animation est en cours, par exemple).

Remarques

> Étant donné que l'implémentation du plateau vous appartient, l'interface graphique ne sait pas comment l'interpréter directement. C'est pourquoi vous devez écrire des fonctions que l'interface graphique appellera pour récupérer les informations du plateau.

Définition à la ligne 115 du fichier game.h.

3.1.2 Documentation des champs

3.1.2.1 dummy

```
int dummy
```

Champ factice pour éviter une erreur de structure vide à la compilation. Doit être remplacé par votre implémentation.

Définition à la ligne 116 du fichier game.h.

La documentation de cette structure a été générée à partir du fichier suivant :

— game.h

3.2 Référence de la structure cell

Structure représentant une case. Utilisée pour communiquer le contenu d'une case à l'interface graphique.

```
#include <game.h>
```

Champs de données

— element object

Type de la gemme ou de l'élément.

— union {
  gem_power power
  short countdown
};

3.2.1 Description détaillée

Structure représentant une case. Utilisée pour communiquer le contenu d'une case à l'interface graphique.

Remarques

Cette structure est utilisée pour transmettre des informations sur une case à l'interface graphique afin qu'elle puisse afficher les visuels appropriés. Cet encodage de l'information de la case est spécifique à l'interface graphique. La façon dont vous choisissez de stocker les informations de la case en interne dans la logique du jeu vous appartient. Vous devrez convertir entre votre représentation interne et cette structure lors de la communication avec l'interface graphique qui se fait par la fonction board_get_cell définie ci-dessous.

Définition à la ligne 186 du fichier game.h.

3.2.2 Documentation des champs

3.2.2.1 object

```
element object
```

Type de la gemme ou de l'élément.

Définition à la ligne 187 du fichier game.h.

3.2.2.2 power

```
gem_power power
```

Pouvoir pour les gemmes (non pertinent pour les non-gemmes).

Définition à la ligne 189 du fichier game.h.

3.2.2.3 countdown

```
short countdown
```

Compte à rebours pour les bombes (non pertinent pour les non-bombes).

Définition à la ligne 191 du fichier game.h.

3.2.2.4 [union]

```
union { ... }
```

La documentation de cette structure a été générée à partir du fichier suivant :

— game.h

3.3 Référence de la structure game

Structure représentant un jeu complet.

```
#include <game.h>
```

Champs de données

— board * b

Pointeur vers le plateau de jeu.

— bool puzzle

Mode de jeu (true : mode puzzle, false : mode classique).

— bool lines

— bool upgrades

— short gravity_control

— bool nodelay

— bool easy

— bool gravity_gems

— bool bombs

Indicateur indiquant que le jeu peut générer des bombes.

— short refill_mode

— score_block * score

3.3.1 Description détaillée

Structure représentant un jeu complet.

Cette structure contient toutes les informations sur un jeu en cours. Elle inclut le plateau de jeu, les paramètres du jeu (mode, options) et le score.

Remarques

Le paramètre le plus important est le booléen "puzzle" qui indique si le jeu est en mode puzzle (true) ou en mode classique (false). En fonction de ce paramètre, certains autres champs peuvent ne pas être pertinents et doivent être définis à des valeurs spécifiques.

Les paramètres communs aux deux modes sont :

- lines : si le jeu utilise des zones destructibles basées sur les lignes ou les composantes fortement connexes (SCC),
- upgrades : si le jeu peut améliorer les gemmes (en gemmes enflammées, gemmes étoilées et hypercubes),
- gravity_control : comment le joueur peut contrôler la gravité (0 : ne peut pas changer, 1 : rotation horaire, 2 : libre).

Les paramètres spécifiques au mode classique sont (nodelay doit toujours être à true en mode puzzle) :

- nodelay : si le jeu est en mode nodelay. true : les gemmes sont immédiatement détruites (toujours à cette valeur en mode puzzle), false : destruction sur demande,
- easy : si le jeu est en mode facile (de nouvelles gemmes sont générées de sorte qu'une solution existe toujours),
- gravity_gems : si le jeu peut générer des gemmes de gravité,
- bombs : si le jeu peut générer des bombes.

L'unique paramètre spécifique au mode puzzle spécifie comment la file de stockage est utilisée (il doit être défini à "pas de file" (valeur 2) en mode classique) :

- refill_mode : le mode de stockage utilisé en mode puzzle (0 : stocke les gemmes éliminées, 1 : file prédéterminée de rochers et bombes, 2 : Pas de file de stockage).

Définition à la ligne 333 du fichier game.h.

3.3.2 Documentation des champs

3.3.2.1 b

```
board* b
```

Pointeur vers le plateau de jeu.

Définition à la ligne 335 du fichier game.h.

3.3.2.2 puzzle

```
bool puzzle
```

Mode de jeu (true : mode puzzle, false : mode classique).

Définition à la ligne 338 du fichier game.h.

3.3.2.3 lines

```
bool lines
```

Spécification des zones destructibles (true : lignes d'au moins 3, false : SCCs de taille au moins 4).

Définition à la ligne 341 du fichier game.h.

3.3.2.4 upgrades

`bool upgrades`

Indicateur indiquant que le jeu peut améliorer les gemmes (en gemmes enflammées, gemmes étoilées et hypercubes).

Définition à la ligne 344 du fichier game.h.

3.3.2.5 gravity_control

`short gravity_control`

Contrôle de la gravité disponible pour le joueur (0 : ne peut pas changer, 1 : rotation horaire, 2 : libre).

Définition à la ligne 346 du fichier game.h.

3.3.2.6 nodelay

`bool nodelay`

Mode d'élimination (true : les gemmes sont immédiatement retirées après chaque échange, false : retrait sur demande de détonation).

Définition à la ligne 351 du fichier game.h.

3.3.2.7 easy

`bool easy`

Indicateur indiquant que le jeu est en mode facile (les gemmes sont générées de sorte qu'une solution existe toujours).

Définition à la ligne 354 du fichier game.h.

3.3.2.8 gravity_gems

`bool gravity_gems`

Indicateur indiquant que le jeu peut générer des gemmes de gravité.

Définition à la ligne 357 du fichier game.h.

3.3.2.9 bombs

```
bool bombs
```

Indicateur indiquant que le jeu peut générer des bombes.

Définition à la ligne 359 du fichier game.h.

3.3.2.10 refill_mode

```
short refill_mode
```

File de stockage utilisée en mode puzzle (0 : stocke les gemmes éliminées, 1 : file prédéterminée de rochers et bombes, 2 : Pas de file de stockage).

Définition à la ligne 363 du fichier game.h.

3.3.2.11 score

```
score_block* score
```

Pointeur vers le bloc de score. La plupart des champs ne sont significatifs qu'en mode classique, mais certains (comme le nombre de coups) sont également utilisés en mode puzzle.

Définition à la ligne 368 du fichier game.h.

La documentation de cette structure a été générée à partir du fichier suivant :

— game.h

3.4 Référence de la structure hash_table

Solveur pour le mode puzzle.

```
#include <game_solver.h>
```

Champs de données

— ulong hash_capacity

Capacité actuelle de la table de hachage.

— ulong hash_size

Taille de la table (nombre d'éléments présents).

— ulong * hash_table

Pointeur vers la table elle-même (tableau d'indices vers le stockage externe).

— int element_size

Nombre d'entiers utilisés pour encoder un élément dans le stockage externe.

3.4.1 Description détaillée

Solveur pour le mode puzzle.

Ce module fournit des fonctionnalités pour résoudre les puzzles à l'aide de divers algorithmes de parcours de graphe (DFS, BFS et A*). Une implémentation de table de hachage est également fournie pour stocker efficacement les configurations de plateau visitées par ces algorithmes. Elle est déjà programmée et vous NE DEVEZ PAS LA MODIFIER. Les seules fonctions à implémenter sont solve_puzzle() et game_implement_solver_move() à la fin de ce fichier (vous pouvez bien sûr ajouter vos propres fonctions utilitaires).

Remarques

> La table de hachage ne peut stocker que des éléments encodés comme des tableaux d'entiers. Cela signifie que pour stocker des configurations de plateau, vous devez d'abord les encoder sous forme de tableaux d'entiers. De plus, la table de hachage ne stocke pas les éléments eux-mêmes, mais des indices vers un tableau de stockage externe que vous devez gérer vous-même.

Structure de table de hachage.

Utilisée pour stocker les configurations de plateau visitées par le solveur. La table de hachage ne stocke pas les configurations elles-mêmes, mais des indices vers un tableau de stockage externe passé aux fonctions de la table (nécessaire car ce tableau de stockage peut être réalloué et déplacé en mémoire).

Remarques

> Chaque élément inséré doit être encodé dans le tableau externe comme une séquence d'entiers (de longueur fixe, définie à la création de la table). En particulier, les configurations de plateau doivent être encodées sous forme de tableaux d'entiers.

Définition à la ligne 49 du fichier game_solver.h.

3.4.2 Documentation des champs

3.4.2.1 hash_capacity

```
ulong hash_capacity
```

Capacité actuelle de la table de hachage.

Définition à la ligne 51 du fichier game_solver.h.

3.4.2.2 hash_size

```
ulong hash_size
```

Taille de la table (nombre d'éléments présents).

Définition à la ligne 52 du fichier game_solver.h.

3.4.2.3 hash_table

```
ulong* hash_table
```

Pointeur vers la table elle-même (tableau d'indices vers le stockage externe).

Définition à la ligne 53 du fichier game_solver.h.

3.4.2.4 element_size

```
int element_size
```

Nombre d'entiers utilisés pour encoder un élément dans le stockage externe.

Définition à la ligne 54 du fichier game_solver.h.

La documentation de cette structure a été générée à partir du fichier suivant :

— game_solver.h

3.5 Référence de la structure score_block

Structure pour contenir les scores d'un jeu.

```
#include <game.h>
```

Champs de données

— int level

Niveau actuel (spécifique au mode classique).

— int level_score

— int target_score

— int scale_score

— int scores [SC_SIZE]

Tableau pour contenir diverses métriques de score.

— int refills

3.5.1 Description détaillée

Structure pour contenir les scores d'un jeu.

Remarques

La plupart des champs ne sont significatifs qu'en mode classique, mais certains sont également utilisés en mode puzzle.

Définition à la ligne 278 du fichier game.h.

3.5.2 Documentation des champs

3.5.2.1 level

```
int level
```

Niveau actuel (spécifique au mode classique).

Définition à la ligne 279 du fichier game.h.

3.5.2.2 level_score

```
int level_score
```

Score depuis le début du niveau actuel (utilisé pour déterminer quand le niveau est terminé).

Définition à la ligne 280 du fichier game.h.

3.5.2.3 target_score

`int target_score`

Score cible à atteindre pour terminer le niveau actuel.

Définition à la ligne 282 du fichier game.h.

3.5.2.4 scale_score

`int scale_score`

Pourcentage d'augmentation du score cible (appliqué après chaque niveau terminé).

Définition à la ligne 284 du fichier game.h.

3.5.2.5 scores

`int scores[SC_SIZE]`

Tableau pour contenir diverses métriques de score.

Définition à la ligne 286 du fichier game.h.

3.5.2.6 refills

`int refills`

Nombre de fois qu'un élément a été défilé de la file de stockage (spécifique au mode puzzle).

Définition à la ligne 287 du fichier game.h.

La documentation de cette structure a été générée à partir du fichier suivant :

— game.h

3.6 Référence de la structure score_table

Structure permettant de stocker le tableau des scores affiché par l'interface graphique (GUI).

`#include <game_scores.h>`

Champs de données

— char(* names)[64]

Pointeur vers un tableau de noms de joueurs.

— int ** scores

Pointeur vers un tableau de scores.

— int size

— int player_index

Indice du score du joueur actuel (-1 si absent).

— int * permutation

3.6.1 Description détaillée

Structure permettant de stocker le tableau des scores affiché par l'interface graphique (GUI).

Cette structure est utilisée dans trois situations spécifiques :

- Lors de l'affichage des meilleurs scores dans le menu. Dans ce cas, elle est remplie uniquement avec les scores chargés depuis le fichier de scores.
- Lors de l'affichage du score à la fin d'un niveau dans une partie classique. Dans ce cas, elle contient les meilleurs scores chargés depuis le fichier de scores ainsi que le score du joueur actuel.
- Lors de l'affichage du score final à la fin d'une partie classique. Dans ce cas, elle contient les meilleurs scores chargés depuis le fichier de scores ainsi que le score du joueur actuel.

Remarques

Le tableau des noms de joueurs contient des chaînes de caractères d'une longueur maximale de 64 (y compris le caractère nul de fin), ce qui signifie que les noms de joueurs peuvent comporter au maximum 63 caractères.

Définition à la ligne 54 du fichier game_scores.h.

3.6.2 Documentation des champs

3.6.2.1 names

`char(* names)[64]`

Pointeur vers un tableau de noms de joueurs.

Définition à la ligne 55 du fichier game_scores.h.

3.6.2.2 scores

`int** scores`

Pointeur vers un tableau de scores.

Définition à la ligne 56 du fichier game_scores.h.

3.6.2.3 size

`int size`

Nombre d'entrées dans la table des scores (taille des tableaux des noms et des scores).

Définition à la ligne 57 du fichier game_scores.h.

3.6.2.4 player_index

`int player_index`

Indice du score du joueur actuel (-1 si absent).

Définition à la ligne 59 du fichier game_scores.h.

3.6.2.5 permutation

`int* permutation`

Pointeur vers un tableau représentant l'ordre trié des scores.

Définition à la ligne 60 du fichier game_scores.h.

La documentation de cette structure a été générée à partir du fichier suivant :

— game_scores.h

3.7 Référence de la structure solver_move

Structure récursive représentant une séquence de coups d'une solution de puzzle.

`#include <game_solver.h>`

Champs de données

— solver_move_type type

Type du coup.

— int index

Pour un échange, indice de la première case. Pour un remplissage, indice où l'élément est placé. Sans objet sinon.

— direction dir

Pour un échange, direction (RIGHT ou DOWN). Pour la gravité, nouvelle direction. Sans objet sinon.

— struct solver_move * next

Pointeur vers le prochain coup dans la liste.

3.7.1 Description détaillée

Structure récursive représentant une séquence de coups d'une solution de puzzle.

Définition à la ligne 112 du fichier game_solver.h.

3.7.2 Documentation des champs

3.7.2.1 type

```
solver_move_type type
```

Type du coup.

Définition à la ligne 114 du fichier game_solver.h.

3.7.2.2 index

```
int index
```

Pour un échange, indice de la première case. Pour un remplissage, indice où l'élément est placé. Sans objet sinon.

Définition à la ligne 115 du fichier game_solver.h.

3.7.2.3 dir

```
direction dir
```

Pour un échange, direction (RIGHT ou DOWN). Pour la gravité, nouvelle direction. Sans objet sinon.

Définition à la ligne 116 du fichier game_solver.h.

3.7.2.4 next

```
struct solver_move* next
```

Pointeur vers le prochain coup dans la liste.

Définition à la ligne 117 du fichier game_solver.h.

La documentation de cette structure a été générée à partir du fichier suivant :

— game_solver.h

Chapitre 4

Documentation des fichiers

4.1 Référence du fichier alloc.h

Macros et fonctions d'allocation mémoire.

```
#include "error.h"
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
```

Macros

- #define CREATE_MEMORY(p, num_objects)

  Macro pour allouer de la mémoire en utilisant la fonction malloc().

- #define CREATE_ZERO_MEMORY(p, num_objects)

  Macro pour allouer de la mémoire en utilisant la fonction calloc().

- #define REDIMENSION_MEMORY(p, num_objects)

  Macro pour réallouer de la mémoire en utilisant la fonction realloc().

- #define CHECK_NULL(n, ...)

  Macro permettant de tester si un ou plusieurs pointeurs sont NULL.

Fonctions

— void check_null (const char *function, char *file, int line, int n,...)

Ne pas utiliser cette fonction directement, mais à la place la macro CHECK_NULL(), qui indique dans le message la position de l'erreur dans le source.

— void multiple_free (void *p,...)

Fonction pour libérer de la mémoire en utilisant la fonction free().

4.1.1 Description détaillée

Macros et fonctions d'allocation mémoire.

Les macros CREATE_MEMORY(), CREATE_ZERO_MEMORY() et REDIMENSION_MEMORY() définies dans ce fichier permettent d'allouer de la mémoire dynamiquement, en vérifiant que l'allocation a réussi. En cas d'échec, elles affichent un message d'erreur sur la sortie erreur, en indiquant la fonction, le fichier et la ligne où l'allocation a échoué, et quittent le programme.

La fonction multiple_free() permet de libérer plusieurs pointeurs en un seul appel (mais attention, aucun de ces pointeurs ne doit être `NULL`, et tous doivent avoir été alloués dynamiquement avant cette libération).

La macro CHECK_NULL() permet de vérifier qu'un ou plusieurs pointeurs ne sont pas `NULL`.

La fonction check_null() n'est pas destinée à être utilisée directement, mais est utilisée par la macro CHECK_NULL().

Définition dans le fichier alloc.h.

4.1.2 Documentation des macros

4.1.2.1 CREATE_MEMORY

```
#define CREATE_MEMORY(
            p,
            num_objects)
```

Valeur :

```
    do {                                                                    \
        DEBUG("Allocating  %lu objects.", (unsigned long)(num_objects));    \
        void *tmp = malloc(sizeof *(p) * (num_objects));                    \
        if (!tmp)                                                           \
            FATAL("Malloc error.");                                         \
        (p) = tmp;                                                          \
    } while (0)
```

Macro pour allouer de la mémoire en utilisant la fonction malloc().

Paramètres

p	Pointeur à allouer.
num_objects	Nombre d'objets à allouer.

Alloue de la mémoire permettant de stocker `num_objects`, chacun de type `*p`.

Exemple d'utilisation :

```
backmove *p;
CREATE_MEMORY(p, 10); // Alloue un tableau de 10 backmove.
```

Si l'allocation échoue, affiche un message d'erreur et quitte le programme.

Attention

Le pointeur p doit avoir été déclaré, mais ne pas pointer vers une zone déjà allouée dynamiquement (sous peine de créer une fuite mémoire). Si p pointe déjà vers une zone allouée dynamiquement, il faut utiliser REDIMENSION_MEMORY.

Voir également

REDIMENSION_MEMORY()

Définition à la ligne 57 du fichier alloc.h.

4.1.2.2 CREATE_ZERO_MEMORY

```
#define CREATE_ZERO_MEMORY(
            p,
            num_objects)
```

Valeur :

```
    do {                                                                    \
        DEBUG("Callocating  %lu objects.", (unsigned long)(num_objects));   \
        void *tmp = calloc((num_objects), sizeof *(p));                     \
        if (!tmp)                                                           \
            FATAL("Calloc error.");                                         \
        (p) = tmp;                                                          \
    } while (0)
```

Macro pour allouer de la mémoire en utilisant la fonction calloc().

Paramètres

p	Pointeur à allouer.
num_objects	Nombre d'objets à allouer.

Alloue de la mémoire initialisée à 0, permettant de stocker `num_objects`, chacun de type `*p`.

Exemple d'utilisation :

```
backmove *p;
CREATE_ZERO_MEMORY(p, 10); // Alloue un tableau de 10 backmove initialisé à 0
```

Si l'allocation échoue, affiche un message d'erreur et quitte le programme.

Attention

Le pointeur p doit avoir été déclaré, mais ne pas pointer vers une zone déjà allouée dynamiquement (sous peine de créer une fuite mémoire). Si p pointe déjà vers une zone allouée dynamiquement, il faut utiliser REDIMENSION_MEMORY.

Voir également

REDIMENSION_MEMORY()

Définition à la ligne 92 du fichier alloc.h.

4.1.2.3 REDIMENSION_MEMORY

```
#define REDIMENSION_MEMORY(
            p,
            num_objects)
```

Valeur :

```
    do {                                                                    \
        DEBUG("Reallocating  %lu objects.", (unsigned long)(num_objects));  \
        void *tmp = realloc((p), sizeof *(p) * (num_objects));              \
        if (!tmp) {                                                         \
            FATAL("Realloc error.");                                        \
        }                                                                   \
        (p) = tmp;                                                          \
    } while (0)
```

Macro pour réallouer de la mémoire en utilisant la fonction realloc().

Paramètres

p	Pointeur à allouer.
num_objects	Nouveau nombre d'objets à allouer.

Ré-alloue la mémoire pointée par un pointeur p, dont la valeur doit avoir été retournée précédemment par une fonction d'allocation dynamique (malloc, calloc, realloc ou une des macros de ce fichier).

Si l'allocation échoue, affiche un message d'erreur et quitte le programme.

Exemple d'utilisation :

```
backmove *p;
CREATE_MEMORY(p, 10); // Alloue un tableau de 10 backmove.
                      // ...
REDIMENSION_MEMORY(p, 20); // Ré-alloue un tableau de 20 backmove en copiant
                           // le tableau précédent en début ce nouveau
                           // tableau, et affecte à p l'adresse de la
                           // première case de ce tableau.
```

Si l'allocation échoue, affiche un message d'erreur et quitte le programme.

Attention

Le pointeur p doit avoir été déclaré et pointer vers une zone déjà allouée dynamiquement.

Voir également

CREATE_MEMORY(), CREATE_ZERO_MEMORY()

Définition à la ligne 132 du fichier alloc.h.

4.1.2.4 CHECK_NULL

```
#define CHECK_NULL(
            n,
            ...)
```

Valeur :

```
check_null(__func__, __FILE__, __LINE__, n __VA_OPT__(, ) __VA_ARGS__)
```

Macro permettant de tester si un ou plusieurs pointeurs sont NULL.

Paramètres

n	Nombre de pointeurs à tester.
...	Pointeurs à tester, en alternance avec des chaînes de caractères.

Il doit y avoir un pointeur à tester pour chaque chaîne de caractères, et inversement.

La chaîne de caractères qui suit chaque pointeur permet de personnaliser le message d'erreur.

Exemple d'utilisation :

```
CHECK_NULL(3, p1, "Name1", p2, "Name2", p3, "Name3");
```

Ici, 3 est le nombre de pointeurs à vérifier, et p1, p2 et p3 sont les pointeurs. Si p1 et p3 sont NULL, la macro affichera un message d'erreur de la forme :

[CRITICAL] Name1 is NULL!
[CRITICAL] Name3 is NULL!

Voir également

check_null()

Définition à la ligne 168 du fichier alloc.h.

4.1.3 Documentation des fonctions

4.1.3.1 check_null()

```
void check_null (
           const char * function,
           char * file,
           int line,
           int n,
            ...)
```

Ne pas utiliser cette fonction directement, mais à la place la macro CHECK_NULL(), qui indique dans le message la position de l'erreur dans le source.

Voir également

CHECK_NULL()

4.1.3.2 multiple_free()

```
void multiple_free (
           void * p,
            ...)
```

Fonction pour libérer de la mémoire en utilisant la fonction free().

Appelle free sur un nombre variable d'arguments. Le dernier argument doit être NULL.

Exemple d'utilisation :

```
int *p1, *p2, *p3;

CREATE_MEMORY(p1, 10); // Alloue un tableau de 10 int.
CREATE_MEMORY(p2, 10);
CREATE_MEMORY(p3, 10);
// ...
multiple_free(p1, p2, p3, NULL);
```

Attention

Cette fonction est dangereuse et il faut savoir ce que vous faites si vous l'utilisez ! Les pointeurs à libérer **ne doivent pas** être NULL. Dans l'exemple précédent, si p2 est NULL, on ne libère pas p3 et cela peut créer une fuite mémoire.

4.2 alloc.h

Aller à la documentation de ce fichier.

```
00001
00022
00023 #ifndef ALLOC_H_
00024 #define ALLOC_H_
00025
00026 #include "error.h"
00027 #include <stdarg.h>
00028 #include <stdio.h>
00029 #include <stdlib.h>
00030
00056
00057 #define CREATE_MEMORY(p, num_objects)                                          \
00058     do {                                                                       \
00059         DEBUG("Allocating  %lu objects.", (unsigned long)(num_objects));       \
00060         void *tmp = malloc(sizeof *(p) * (num_objects));                       \
00061         if (!tmp)                                                              \
00062             FATAL("Malloc error.");                                            \
00063         (p) = tmp;                                                             \
00064     } while (0)
00065
00091
00092 #define CREATE_ZERO_MEMORY(p, num_objects)                                     \
00093     do {                                                                       \
00094         DEBUG("Callocating  %lu objects.", (unsigned long)(num_objects));      \
00095         void *tmp = calloc((num_objects), sizeof *(p));                        \
00096         if (!tmp)                                                              \
00097             FATAL("Calloc error.");                                            \
00098         (p) = tmp;                                                             \
00099     } while (0)
00100
00131
00132 #define REDIMENSION_MEMORY(p, num_objects)                                     \
00133     do {                                                                       \
00134         DEBUG("Reallocating  %lu objects.", (unsigned long)(num_objects));     \
00135         void *tmp = realloc((p), sizeof *(p) * (num_objects));                 \
00136         if (!tmp) {                                                            \
00137             FATAL("Realloc error.");                                           \
00138         }                                                                      \
00139         (p) = tmp;                                                             \
00140     } while (0)
00141
00167
00168 #define CHECK_NULL(n, ...)                                                     \
00169     check_null(__func__, __FILE__, __LINE__, n __VA_OPT__(, ) __VA_ARGS__)
00170
00178
00179 void check_null(const char *function, char *file, int line, int n, ...);
00180
00203
00204 void multiple_free(void *p, ...);
00205
00206 #endif // ALLOC_H_
```

4.3 Référence du fichier error.h

Macros permettant d'afficher des messages d'erreur.

Macros

— #define ATTENTION "\xe2\x9d\x97"

Symbole ❗.

— #define HINT "\xe2\x9c\xa8"

Symbole ✨.

— #define OK "\xE2\x9C\x85"

Symbole ✅.

— #define KO "\xE2\x9D\x8C"

Symbole ❌.

— #define DEBUG_LEVEL INFO

Niveau de debug, INFO par défaut.

— #define PRINT_ERROR(symbol, name, function, file, line, msg, ...)

Affiche un message d'erreur sur la sortie d'erreur.

— #define FATAL(msg, ...)

Affiche un message de niveau FATAL et termine le programme.

— #define PRINT_DEBUG(symbol, name, level, msg, ...)

— #define TRACE(msg, ...)

Affiche un message de niveau TRACE.

— #define DEBUG(msg, ...)

Affiche un message de niveau DEBUG.

— #define INFO(msg, ...)

Affiche un message de niveau INFO.

— #define WARNING(msg, ...)

Affiche un message de niveau WARNING.

— #define ERROR(msg, ...)

Affiche un message de niveau ERROR.

— #define CRITICAL(msg, ...)

Affiche un message de niveau CRITICAL.

Énumérations

— enum debug_level {
OFF , CRITICAL , ERROR , WARNING ,
INFO , DEBUG , TRACE }

— enum msg_level {
CRITICAL_L , ERROR_L , WARNING_L , INFO_L ,
DEBUG_L , TRACE_L }

4.3.1 Description détaillée

Macros permettant d'afficher des messages d'erreur.

Le fichier fournit les macros TRACE(), DEBUG(), INFO(), WARNING(), ERROR(), CRITICAL() et FATAL(), qui permettent d'afficher des messages d'erreur sur la sortie d'erreur, et, pour la macro FATAL(), de terminer le programme. Les messages d'erreur sont affichés avec le nom de la fonction, le nom du fichier et le numéro de la ligne où l'appel à la macro est effectué.

Les autres macros du fichier sont essentiellement destinées à implémenter les macros TRACE(), DEBUG(), INFO(), WARNING(), ERROR(), CRITICAL() et FATAL(). Elles ne sont pas destinées à être utilisées directement par l'utilisateur (mais rien ne l'empêche).

Les macros TRACE(), DEBUG(), INFO(), WARNING(), ERROR(), CRITICAL() et FATAL() prennent en argument un message à afficher, qui peut contenir des spécificateurs de conversion comme dans la fonction printf. Elles peuvent prendre des arguments supplémentaires, un par spécificateur de conversion.

Exemple d'utilisation :

```
#include "error.h"
ERROR("La variable colors vaut %d, elle devrait être supérieure à %d.",
       colors, MIN_COLORS);
```

Note

Les messages d'un certain type sont affichés en fonction du niveau de debug, fixé par la variable DEBUG_LEVEL. Par exemple, si le niveau de debug est à ERROR, seules les macros ERROR(), CRITICAL() et FATAL() afficheront leur message. Les autres macros n'afficheront rien. Par défaut, le niveau de debug est INFO.

Niveaux de debug :

- OFF (seulement les messages FATAL sont affichés),
- CRITICAL (seulement les messages CRITICAL et FATAL),
- ERROR (seulement les messages ERROR, CRITICAL et FATAL),
- WARNING (seulement les messages WARNING, ERROR, CRITICAL et FATAL),
- INFO (tous les messages sauf DEBUG et TRACE),
- DEBUG (tous les messages sauf TRACE),
- TRACE (tous les messages).

Pour changer le niveau de debug, il faut définir la macro DEBUG_LEVEL avant d'inclure error.h, typiquement à la compilation. Par exemple, pour afficher tous les messages de debug, il faut définir DEBUG_LEVEL à TRACE. Pour cela, on peut compiler le programme ainsi :

```
make -B CFLAGS="-DDEBUG_LEVEL=TRACE"
```

Voir également

DEBUG_LEVEL et les macros TRACE(), DEBUG(), INFO(), WARNING(), ERROR(), CRITICAL() et FATAL().

Définition dans le fichier error.h.

4.3.2 Documentation des macros

4.3.2.1 ATTENTION

```
#define ATTENTION "\xe2\x9d\x97"
```

Symbole ❗.

Définition à la ligne 60 du fichier error.h.

4.3.2.2 HINT

```
#define HINT "\xe2\x9c\xa8"
```

Symbole ✨.

Définition à la ligne 62 du fichier error.h.

4.3.2.3 OK

```
#define OK "\xE2\x9C\x85"
```

Symbole ✅.

Définition à la ligne 64 du fichier error.h.

4.3.2.4 KO

```
#define KO "\xE2\x9D\x8C"
```

Symbole ❌.

Définition à la ligne 66 du fichier error.h.

4.3.2.5 DEBUG_LEVEL

```
#define DEBUG_LEVEL INFO
```

Niveau de debug, INFO par défaut.

Par défaut, le niveau de debug est INFO. Pour changer le niveau de debug, il faut définir la macro DEBUG_LEVEL avant d'inclure error.h.

Cela peut se faire au moment de la compilation :
`make -B CFLAGS="-DDEBUG_LEVEL=WARNING"`

Par exemple, pour afficher tous les messages de debug, il faut définir DEBUG_LEVEL à TRACE. Pour n'afficher que les messages de niveau FATAL, il faut définir DEBUG_LEVEL à OFF.

Niveaux possibles :

- OFF (seulement les messages FATAL sont affichés),
- CRITICAL (seulement les messages CRITICAL et FATAL),
- ERROR (seulement les messages ERROR, CRITICAL et FATAL),
- WARNING (seulement les messages WARNING, ERROR, CRITICAL et FATAL),
- INFO (tous les messages sauf DEBUG et TRACE),
- DEBUG (tous les messages sauf TRACE),
- TRACE (tous les messages).

Voir également

enum debug_level

Définition à la ligne 103 du fichier error.h.

4.3.2.6 PRINT_ERROR

```
#define PRINT_ERROR(
            symbol,
            name,
            function,
            file,
            line,
            msg,
            ...)
```

Valeur :

```
do {                                                                \
  fprintf(stderr,                                                   \
          "\n" symbol                                               \
          " ["                                                      \
          name                                                      \
          "]"                                                       \
          " - Function %s (%s:%d) -\n  "                            \
          msg "\n",                                                 \
          function, file, line                                      \
          __VA_OPT__( , ) __VA_ARGS__);                             \
} while (0)
```

Affiche un message d'erreur sur la sortie d'erreur.

Cette macro est utilisée par les macros TRACE, DEBUG, INFO, WARNING, ERROR, CRITICAL et FATAL.

Paramètres

symbol	Symbole à afficher (ATTENTION, HINT, OK ou KO).
name	Nom du message (comme "TRACE", "DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL" ou "FATAL").
function	Nom de la fonction où le message est affiché.
file	Nom du fichier où le message est affiché.
line	Numéro de la ligne où le message est affiché.
msg	Message à afficher. Ce message peut contenir les mêmes spécificateurs de conversion que la fonction printf (comme `d`, `s`, etc.).
...	Arguments supplémentaires à afficher avec le message, un par spécificateur de conversion.

Définition à la ligne 125 du fichier error.h.

4.3.2.7 FATAL

```
#define FATAL(
            msg,
            ...)
```

Valeur :

```
do {                                                    \
  PRINT_ERROR(KO, "FATAL", __func__, __FILE__, __LINE__, \
              msg __VA_OPT__( , ) __VA_ARGS__);          \
  exit(EXIT_FAILURE);                                   \
} while (0)
```

Affiche un message de niveau FATAL et termine le programme.

Définition à la ligne 140 du fichier error.h.

4.3.2.8 PRINT_DEBUG

```
#define PRINT_DEBUG(
            symbol,
            name,
            level,
            msg,
            ...)
```

Valeur :

```
do {                                                      \
  if ((int)DEBUG_LEVEL > (int)level)                      \
  PRINT_ERROR(symbol, name, __func__, __FILE__, __LINE__, \
              msg __VA_OPT__( , ) __VA_ARGS__);           \
} while (0)
```

Définition à la ligne 147 du fichier error.h.

4.3.2.9 TRACE

```
#define TRACE(
            msg,
            ...)
```

Valeur :

```
PRINT_DEBUG(OK, "TRACE", TRACE_L, \
            msg, __VA_ARGS__)
```

Affiche un message de niveau TRACE.

Définition à la ligne 155 du fichier error.h.

4.3.2.10 DEBUG

```
#define DEBUG(
            msg,
            ...)
```

Valeur :

```
PRINT_DEBUG(HINT, "DEBUG", DEBUG_L, \
            msg, __VA_ARGS__)
```

Affiche un message de niveau DEBUG.

Définition à la ligne 160 du fichier error.h.

4.3.2.11 INFO

```
#define INFO(
            msg,
            ...)
```

Valeur :

```
PRINT_DEBUG(HINT, "INFO", INFO_L, \
            msg, __VA_ARGS__)
```

Affiche un message de niveau INFO.

Définition à la ligne 165 du fichier error.h.

4.3.2.12 WARNING

```
#define WARNING(
            msg,
            ...)
```

Valeur :

```
PRINT_DEBUG(ATTENTION, "WARNING", WARNING_L, \
            msg, __VA_ARGS__)
```

Affiche un message de niveau WARNING.

Définition à la ligne 170 du fichier error.h.

4.3.2.13 ERROR

```
#define ERROR(
            msg,
            ...)
```

Valeur :

```
PRINT_DEBUG(ATTENTION, "ERROR", ERROR_L, \
            msg, __VA_ARGS__)
```

Affiche un message de niveau ERROR.

Définition à la ligne 175 du fichier error.h.

4.3.2.14 CRITICAL

```
#define CRITICAL(
            msg,
            ...)
```

Valeur :

```
PRINT_DEBUG(KO, "CRITICAL", CRITICAL_L, \
            msg, __VA_ARGS__)
```

Affiche un message de niveau CRITICAL.

Définition à la ligne 180 du fichier error.h.

4.3.3 Documentation du type de l'énumération

4.3.3.1 debug_level

```
enum debug_level
```

Valeurs énumérées

OFF	
CRITICAL	
ERROR	
WARNING	
INFO	
DEBUG	
TRACE	

Définition à la ligne 70 du fichier error.h.

4.3.3.2 msg_level

```
enum msg_level
```

Valeurs énumérées

CRITICAL↩ _L	
ERROR_L	
WARNING↩ _L	
INFO_L	
DEBUG_L	
TRACE_L	

Définition à la ligne 71 du fichier error.h.

4.4 error.h

Aller à la documentation de ce fichier.

```
00001
00054
00055 #ifndef ERROR_H_
00056 #define ERROR_H_
00057
00058
00060 #define ATTENTION "\xe2\x9d\x97"
00062 #define HINT "\xe2\x9c\xa8"
00064 #define OK "\xE2\x9C\x85"
00066 #define KO "\xE2\x9D\x8C"
00067
00068 // In X mode, all messages more severe than X are printed.
00069 // clang-format off
00070 enum debug_level {OFF,        CRITICAL, ERROR,     WARNING, INFO,    DEBUG,
      TRACE};
00071 enum msg_level   {CRITICAL_L, ERROR_L,  WARNING_L, INFO_L,  DEBUG_L, TRACE_L};
00072 // clang-format on
00073
00101
00102 #ifndef DEBUG_LEVEL
00103 #define DEBUG_LEVEL INFO
00104 #endif
00105
00106 // __VA_OPTS__ requires C2x. Clang issues spurious warning.
00107
00124
00125 #define PRINT_ERROR(symbol, name, function, file, line, msg, ...) \
00126   do {                                                            \
00127     fprintf(stderr,                                               \
00128             "\n" symbol                                           \
00129             " ["                                                  \
```

```
00130             name                                                  \
00131             "]"                                                   \
00132             " - Function %s (%s:%d) -\n  "                        \
00133             msg "\n",                                             \
00134             function, file, line                                  \
00135             __VA_OPT__( , ) __VA_ARGS__);                         \
00136   } while (0)
00137
00139
00140 #define FATAL(msg, ...)                                          \
00141   do {                                                           \
00142     PRINT_ERROR(KO, "FATAL", __func__, __FILE__, __LINE__, \
00143                 msg __VA_OPT__( , ) __VA_ARGS__);          \
00144     exit(EXIT_FAILURE);                                          \
00145   } while (0)
00146
00147 #define PRINT_DEBUG(symbol, name, level, msg, ...)          \
00148   do {                                                           \
00149     if ((int)DEBUG_LEVEL > (int)level)                           \
00150     PRINT_ERROR(symbol, name, __func__, __FILE__, __LINE__, \
00151                 msg __VA_OPT__( , ) __VA_ARGS__);          \
00152   } while (0)
00153
00155 #define TRACE(msg, ...)             \
00156   PRINT_DEBUG(OK, "TRACE", TRACE_L, \
00157               msg, __VA_ARGS__)
00158
00160 #define DEBUG(msg, ...)               \
00161   PRINT_DEBUG(HINT, "DEBUG", DEBUG_L, \
00162               msg, __VA_ARGS__)
00163
00165 #define INFO(msg, ...)              \
00166   PRINT_DEBUG(HINT, "INFO", INFO_L, \
00167               msg, __VA_ARGS__)
00168
00170 #define WARNING(msg, ...)                        \
00171   PRINT_DEBUG(ATTENTION, "WARNING", WARNING_L, \
00172               msg, __VA_ARGS__)
00173
00175 #define ERROR(msg, ...)                    \
00176   PRINT_DEBUG(ATTENTION, "ERROR", ERROR_L, \
00177               msg, __VA_ARGS__)
00178
00180 #define CRITICAL(msg, ...)                \
00181   PRINT_DEBUG(KO, "CRITICAL", CRITICAL_L, \
00182               msg, __VA_ARGS__)
00183
00184 #endif // ERROR_H_
```

4.5 Référence du fichier game.h

Implémentation du jeu.

```
#include "project_tools.h"
#include <limits.h>
```

```
#include <stdbool.h>
#include <stdlib.h>
```

Structures de données

— struct board

Structure représentant un plateau de jeu.

— struct cell

Structure représentant une case. Utilisée pour communiquer le contenu d'une case à l'interface graphique.

— struct score_block

Structure pour contenir les scores d'un jeu.

— struct game

Structure représentant un jeu complet.

Énumérations

— enum direction {
UP , RIGHT , DOWN , LEFT ,
SIZE_DIRECTION }

Directions à l'intérieur d'un plateau de jeu.

— enum element {
AMBER , AMETHYST , DIAMOND , EMERALD ,
RUBY , SAPPHIRE , TOPAZ , HYPERCUBE ,
ROCK , BOMB , EMPTY , ELEM_SIZE }

Types d'éléments pouvant remplir une case du plateau de jeu.

— enum gem_power {
GEM_BASIC , GEM_GRAVITY , GEM_FLAME , GEM_STAR ,
GEM_SIZE }

Pouvoirs qu'une gemme peut posséder.

— enum score_type {
SC_GLOBAL , SC_LEVEL , SC_SWAPS , SC_CONSUMED ,
SC_FLAMED , SC_STARRED , SC_HYPERCUBES , SC_GRAVITY ,
SC_TIME , SC_SIZE }

Scores enregistrés dans un jeu.

Fonctions

— int board_get_width (board *b)

Retourne la largeur d'un plateau de jeu.

— int board_get_height (board *b)

Retourne la hauteur du plateau de jeu.

— direction board_get_gravity (board *b)

Retourne la direction de la gravité du plateau de jeu.

— bool board_get_picked (board *b, int *i, int *j)

Récupère les coordonnées de la case actuellement sélectionnée dans le plateau (s'il y en a une).

— void board_get_cell (board *b, int i, int j, cell *c)

Récupère le contenu d'une case dans le plateau de jeu.

— int board_refill_size (board *b)

Retourne la taille de la file de stockage dans le plateau (utilisée uniquement en mode puzzle).

— bool board_read_refill (board *b, int k, cell *c)

Lit une case de la file de stockage (sans la retirer).

— game * game_create_classic (int width, int height, bool lines, int target_score, int scale_score, bool nodelay, bool upgrades, bool easy_mode, bool gravity_gems, bool bombs)

Appelée par l'interface pour initialiser un jeu classique avec la largeur et la hauteur spécifiées.

— game * game_create_puzzle (int width, int height, bool lines, bool upgrades, short refill_mode, int ini, bool balanced, short gravity_control)

Appelée par l'interface pour initialiser un jeu puzzle avec la largeur et la hauteur spécifiées.

— void game_next_level (game *g)

Appelée par l'interface lorsqu'un niveau se termine en mode classique pour générer un nouveau plateau de jeu.

— void game_release (game *g)

Appelée par l'interface pour libérer la mémoire allouée pour un jeu.

— game * game_clone (game *g)

Appelée par l'interface pour cloner une structure de jeu.

4.5.1 Description détaillée

Implémentation du jeu.

Définition dans le fichier game.h.

4.5.2 Documentation du type de l'énumération

4.5.2.1 direction

`enum direction`

Directions à l'intérieur d'un plateau de jeu.

Valeurs énumérées

UP	Direction haut.
RIGHT	Direction droite.
DOWN	Direction bas.
LEFT	Direction gauche.
SIZE_DIRECTION	Taille de l'énumération `direction`.

Définition à la ligne 34 du fichier game.h.

4.5.2.2 element

`enum element`

Types d'éléments pouvant remplir une case du plateau de jeu.

Valeurs énumérées

AMBER	Représente une gemme de type AMBRE.
AMETHYST	Représente une gemme de type AMÉTHYSTE.
DIAMOND	Représente une gemme de type DIAMANT.
EMERALD	Représente une gemme de type ÉMERAUDE.
RUBY	Représente une gemme de type RUBIS.
SAPPHIRE	Représente une gemme de type SAPHIR.

TOPAZ	Représente une gemme de type TOPAZE.
HYPERCUBE	Représente un Hypercube.
ROCK	Représente un rocher.
BOMB	Représente une bombe.
EMPTY	Représente une case vide sur le plateau.
ELEM_SIZE	Taille de l'énumération `element`.

Définition à la ligne 46 du fichier game.h.

4.5.2.3 gem_power

`enum gem_power`

Pouvoirs qu'une gemme peut posséder.

Il existe plusieurs types de gemmes avec pouvoirs :

- Gemmes de gravité : lorsqu'elles sont échangées avec un autre élément, elles changent la direction de la gravité vers une direction aléatoire (différente de la direction actuelle).
- Gemmes enflammées : lorsqu'elles sont détruites, elles détruisent également tous leurs voisins dans une zone 3x3.
- Gemmes étoilées : lorsqu'elles sont cassées, elles effacent toute leur ligne et colonne.

Remarques

Dans l'interface graphique, ces paramètres n'affectent que la représentation visuelle de la gemme. Vous devez implémenter le comportement réel des gemmes avec pouvoirs dans la logique du jeu.

Valeurs énumérées

GEM_BASIC	Gemme basique (sans pouvoir).
GEM_GRAVITY	Gemme de gravité.
GEM_FLAME	Gemme enflammée.
GEM_STAR	Gemme étoilée.
GEM_SIZE	Taille de l'énumération `gem_power`.

Définition à la ligne 80 du fichier game.h.

4.5.2.4 score_type

`enum score_type`

Scores enregistrés dans un jeu.

La plupart des valeurs sont inutiles en mode puzzle. Néanmoins, elles sont toujours suivies par souci de simplicité.

Remarques

Le temps est stocké en secondes. Il est géré automatiquement par l'interface graphique. Vous devez l'initialiser à zéro au début du jeu. Il sera mis à jour automatiquement par l'interface graphique.

Valeurs énumérées

SC_GLOBAL	Score global.
SC_LEVEL	Niveau maximum atteint.
SC_SWAPS	Nombre total d'échanges.
SC_CONSUMED	Nombre total d'objets consommés.
SC_FLAMED	Nombre total de gemmes enflammées consommées.
SC_STARRED	Nombre total de gemmes étoilées consommées.
SC_HYPERCUBES	Nombre total d'hypercubes consommés.
SC_GRAVITY	Nombre total de changements de gravité.
SC_TIME	Temps total joué.
SC_SIZE	Taille de l'énumération.

Définition à la ligne 257 du fichier game.h.

4.5.3 Documentation des fonctions

4.5.3.1 board_get_width()

```
int board_get_width (
            board * b)
```

Retourne la largeur d'un plateau de jeu.

Renvoie

La largeur d'un plateau de jeu.

Paramètres

b	Pointeur vers la structure du plateau

4.5.3.2 board_get_height()

```
int board_get_height (
            board * b)
```

Retourne la hauteur du plateau de jeu.

Renvoie

La hauteur du plateau de jeu.

Paramètres

b	Pointeur vers la structure du plateau

4.5.3.3 board_get_gravity()

```
direction board_get_gravity (
            board * b)
```

Retourne la direction de la gravité du plateau de jeu.

Renvoie

La direction de la gravité du plateau de jeu.

Paramètres

b	Pointeur vers la structure du plateau

4.5.3.4 board_get_picked()

```
bool board_get_picked (
            board * b,
            int * i,
            int * j)
```

Récupère les coordonnées de la case actuellement sélectionnée dans le plateau (s'il y en a une).

Renvoie

True si une case est actuellement sélectionnée, false sinon.

Paramètres

b	Pointeur vers la structure du plateau
i	Pointeur pour retourner la coordonnée horizontale de la case sélectionnée dans le plateau (numérotée de gauche à droite).
j	Pointeur pour retourner la coordonnée verticale de la case sélectionnée dans le plateau (numérotée de haut en bas).

4.5.3.5 board_get_cell()

```
void board_get_cell (
            board * b,
            int i,
            int j,
            cell * c)
```

Récupère le contenu d'une case dans le plateau de jeu.

Cette fonction remplit la structure de case fournie avec le contenu de la case située aux coordonnées spécifiées dans le plateau de jeu.

Paramètres

b	Pointeur vers la structure du plateau.
i	Coordonnée horizontale de la case dans le plateau (numérotée de gauche à droite).
j	Coordonnée verticale de la case dans le plateau (numérotée de haut en bas).
c	Pointeur pour stocker la case.

4.5.3.6 board_refill_size()

```
int board_refill_size (
            board * b)
```

Retourne la taille de la file de stockage dans le plateau (utilisée uniquement en mode puzzle).

Renvoie

La taille de la file de stockage, ou -1 s'il n'y a pas de file de stockage

Paramètres

b	Pointeur vers le plateau

4.5.3.7 board_read_refill()

```
bool board_read_refill (
            board * b,
            int k,
            cell * c)
```

Lit une case de la file de stockage (sans la retirer).

Remarques

S'il n'y a pas de file de stockage, la fonction doit retourner false.

Renvoie

true si la lecture a réussi, false sinon.

Paramètres

b	Pointeur vers le plateau
k	Index dans le stockage à lire (0 est le premier élément à être retiré au début de la file).
c	Pointeur pour stocker la case lue du stockage.

4.5.3.8 game_create_classic()

```
game * game_create_classic (
            int width,
            int height,
            bool lines,
            int target_score,
            int scale_score,
            bool nodelay,
            bool upgrades,
            bool easy_mode,
            bool gravity_gems,
            bool bombs)
```

Appelée par l'interface pour initialiser un jeu classique avec la largeur et la hauteur spécifiées.

Alloue de la mémoire pour la structure du jeu et initialise le jeu. Les cases sont toutes remplies aléatoirement avec des types de gemmes. La génération garantit que le jeu est stable : aucune zone destructible n'existe. De plus, elle doit également garantir qu'au moins un coup peut être joué.

Attention

Comme le mode classique n'utilise pas de file de stockage, le paramètre refill_mode doit être défini à 2 (pas de file).

Renvoie

Un pointeur vers la structure de jeu initialisée.

Paramètres

width	Largeur du plateau de jeu.
height	Hauteur du plateau de jeu.
lines	Mode d'élimination du jeu (lignes ou SCCs).
target_score	Score cible à atteindre pour terminer le niveau.
scale_score	Pourcentage de mise à l'échelle du score cible (appliqué après chaque niveau terminé).
nodelay	Indicateur indiquant si le mode nodelay est activé.
upgrades	Indicateur indiquant si les améliorations de gemmes sont activées.
easy_mode	Indicateur indiquant si le mode facile est activé.
gravity_gems	Indicateur indiquant si les gemmes de gravité sont activées.
bombs	Indicateur indiquant si les bombes sont activées.

4.5.3.9 game_create_puzzle()

```
game * game_create_puzzle (
           int width,
           int height,
           bool lines,
           bool upgrades,
           short refill_mode,
           int ini,
           bool balanced,
           short gravity_control)
```

Appelée par l'interface pour initialiser un jeu puzzle avec la largeur et la hauteur spécifiées.

Alloue de la mémoire pour la structure du jeu et initialise le jeu. Les cases sont soit vides, soit remplies aléatoirement avec des types de gemmes. La génération garantit que le jeu est stable :

- aucune zone destructible n'existe, et,
- il n'y a pas de case non vide au-dessus d'une case vide.

Le paramètre "balanced" indique si la génération équilibrée est activée. Lorsqu'elle est activée, le nombre de chaque type de gemme sur le plateau doit être un multiple de 3.

Le paramètre refill_mode spécifie comment la file de stockage doit être initialisée :

- Si refill_mode est 0, la file de stockage est allouée et initialement vide. Les gemmes qui sont éliminées sont stockées dans la file de stockage et peuvent être utilisées pour remplir le plateau plus tard.
- Si refill_mode est 1, la file de stockage est allouée et doit être remplie avec une séquence aléatoire de rochers et de bombes. La longueur de la séquence est fournie par le paramètre ini. Ce paramètre ne doit pas dépasser le nombre total de cases sur le plateau (s'il le fait, il doit être défini à width * height).
- Si refill_mode est 2, la file de stockage n'est pas utilisée et ne doit pas être allouée.

Renvoie

Un pointeur vers la structure de jeu initialisée.

Paramètres

width	Largeur du plateau de jeu.
height	Hauteur du plateau de jeu
lines	Mode d'élimination du jeu (lignes ou SCCs).
upgrades	Indicateur indiquant si les améliorations de gemmes sont activées.
refill_mode	Indicateur indiquant le mode de stockage.
ini	Taille initiale du stockage (pertinent uniquement si refill_mode est défini à 1).
balanced	Indicateur indiquant si la génération équilibrée est activée (le nombre de chaque type de gemme doit être un multiple de 3).
gravity_control	Mode de contrôle de la gravité (0 : ne peut pas changer, 1 : rotation horaire, 2 : libre).

4.5.3.10 game_next_level()

```
void game_next_level (
            game * g)
```

Appelée par l'interface lorsqu'un niveau se termine en mode classique pour générer un nouveau plateau de jeu.

Cette fonction libère le plateau de jeu actuel (mais pas toute la structure du jeu) et génère un nouveau plateau aléatoire avec les mêmes dimensions. Elle met également à jour le bloc de score:

- le niveau est incrémenté de 1,
- le level_score est réinitialisé à 0,
- le target_score est mis à jour en appliquant le pourcentage scale_score.

Remarques

S'il y avait des gemmes avec pouvoirs (enflammée, étoilée, gravité) ou des hypercubes sur le plateau précédent, ils doivent être préservés dans le nouveau plateau. Par exemple, s'il y avait 2 gemmes enflammées sur le plateau précédent, le nouveau plateau doit également contenir exactement 2 gemmes enflammées (elles peuvent être de types différents et placées à des emplacements différents).

Paramètres

g	Pointeur vers la structure du jeu.

4.5.3.11 game_release()

```
void game_release (
            game * g)
```

Appelée par l'interface pour libérer la mémoire allouée pour un jeu.

Cette fonction libère la mémoire utilisée par le plateau de jeu et la structure du jeu elle-même.

4.5.3.12 game_clone()

```
game * game_clone (
           game * g)
```

Appelée par l'interface pour cloner une structure de jeu.

Cette fonction crée une copie complète de la structure de jeu fournie, incluant le plateau de jeu et toutes les données associées.

Remarques

Cette fonction est utilisée par l'interface au début d'un jeu puzzle pour créer une sauvegarde de l'état initial du jeu (ce qui permet de le restaurer).

Renvoie

Un pointeur vers la structure de jeu clonée.

Paramètres

g	Pointeur vers la structure du jeu à cloner

4.6 game.h

Aller à la documentation de ce fichier.

```
00001
00007
00008 #ifndef GAME_FULL_H_
00009 #define GAME_FULL_H_
00010
00011 /*  __  __       _          ____                          */
00012 /* |  \/  | __ _(_)_ __    / ___| __ _ _ __ ___   ___  */
00013 /* | |\/| |/ _` | | '_ \  | |  _ / _` | '_ ` _ \ / _ \ */
00014 /* | |  | | (_| | | | | | | |_| | (_| | | | | | | |  __/ */
00015 /* |_|__|_|\__,_|_|_|_| |_|  \____|\__,_|_| |_| |_|\___| */
00016 /* / ___|| |_ _ __ _   _  ___| |_ _   _ _ __ ___       */
00017 /* \___ \| __| '__| | | |/ __| __| | | | '__/ _ \      */
00018 /*  ___) | |_| |  | |_| | (__| |_| |_| | | |  __/      */
00019 /* |____/ \__|_|   \__,_|\___|\__|\__,_|_|  \___|      */
00020
00021 #include "project_tools.h"
00022 #include <limits.h>
00023 #include <stdbool.h>
00024 #include <stdlib.h>
00025
00026 /***************************/
00027 /*+ Définition du plateau +*/
00028 /***************************/
00029
00034 typedef enum {
```

```
00035     UP,
00036     RIGHT,
00037     DOWN,
00038     LEFT,
00039     SIZE_DIRECTION
00040 } direction;
00041
00046 typedef enum {
00047     AMBER,
00048     AMETHYST,
00049     DIAMOND,
00050     EMERALD,
00051     RUBY,
00052     SAPPHIRE,
00053     TOPAZ,
00054     HYPERCUBE,
00055     ROCK,
00056     BOMB,
00057     EMPTY,
00058     ELEM_SIZE
00059 } element;
00060
00080 typedef enum {
00081     GEM_BASIC,
00082     GEM_GRAVITY,
00083     GEM_FLAME,
00084     GEM_STAR,
00085     GEM_SIZE
00086 } gem_power;
00087
00115 typedef struct {
00116     int dummy;
00118 } board;
00119
00120 /*****************************************************************************/
00121 /*+ Fonctions appelées par l'interface pour récupérer des informations    +*/
00122 /*+ sur un plateau de jeu                                                 +*/
00123 /*****************************************************************************/
00124
00132 int board_get_width(board *b
00133 );
00134
00142 int board_get_height(board *b
00143 );
00144
00152 direction board_get_gravity(board *b
00153 );
00154
00163 bool board_get_picked(
00164     board *b,
00165     int *i,
00167     int *j
00169 );
00170
00186 typedef struct {
00187     element object;
00188     union {
00189         gem_power power;
```

```
00191         short countdown;
00193     };
00194 } cell;
00195
00204 void board_get_cell(board *b,
00205                     int i,
00207                     int j,
00209                     cell *c
00210 );
00211
00220 int board_refill_size(board *b
00221 );
00222
00233 bool board_read_refill(
00234     board *b,
00235     int k,
00237     cell *c
00238 );
00239
00240 /*************************/
00241 /*+ Définition d'un jeu +*/
00242 /*************************/
00243
00257 typedef enum {
00258     SC_GLOBAL,
00259     SC_LEVEL,
00260     SC_SWAPS,
00261     SC_CONSUMED,
00262     SC_FLAMED,
00263     SC_STARRED,
00264     SC_HYPERCUBES,
00265     SC_GRAVITY,
00266     SC_TIME,
00267     SC_SIZE,
00268 } score_type;
00269
00278 typedef struct {
00279     int level;
00280     int level_score;
00282     int target_score;
00284     int scale_score;
00286     int scores[SC_SIZE];
00287     int refills;
00289 } score_block;
00290
00333 typedef struct {
00334     // Plateau de jeu
00335     board *b;
00336
00337     // Le mode de jeu
00338     bool puzzle;
00339
00340     // Paramètres communs aux deux modes
00341     bool lines;
00343     bool
00344         upgrades;
00346     short gravity_control;
00349
```

```
00350     // Paramètres spécifiques au mode classique.
00351     bool nodelay;
00354     bool easy;
00357     bool gravity_gems;
00359     bool bombs;
00360
00361     // Paramètre spécifique au mode puzzle. Doit être défini à "pas de file"
00362     // (valeur 2) en mode classique.
00363     short refill_mode;
00366
00367     // Score
00368     score_block *score;
00372 } game;
00373
00374
      /*****************************************************************************************/
00375 /*+ Fonctions utilisées par l'interface pour créer et mettre à jour un nouveau
00376  * jeu +*/
00377
      /*****************************************************************************************/
00378
00397 game *game_create_classic(
00398     int width,
00399     int height,
00400     bool lines,
00401     int target_score,
00402     int scale_score,
00404     bool nodelay,
00405     bool upgrades,
00407     bool easy_mode,
00408     bool gravity_gems,
00410     bool bombs
00411 );
00412
00447 game *game_create_puzzle(
00448     int width,
00449     int height,
00450     bool lines,
00451     bool upgrades,
00453     short refill_mode,
00454     int ini,
00456     bool balanced,
00459     short gravity_control
00461 );
00462
00484 void game_next_level(game *g
00485 );
00486
00495 void game_release(game *g);
00496
00513 game *game_clone(game *g
00514 );
00515
00516 #endif // GAME_FULL_H_
```

4.7 Référence du fichier game_ias.h

IA pour le mode classique.

`#include "game.h"`

Énumérations

— enum ia_classic { IA_RANDOM , IA_SAFE , IA_AGRESSIVE , IA_CLASSIC_SIZE }

Types d'IA disponibles pour le mode classique.

Fonctions

— bool ia_random (game *g)

Implémentation de l'IA aléatoire (c'est l'IA utilisée dans la démo affichée dans le menu).

— bool ia_safe (game *g)

Implémentation de l'IA prudente (l'IA essaie de jouer des coups plus proches du bord opposé à la gravité).

— bool ia_agressive (game *g)

Implémentation de l'IA agressive (l'IA essaie de jouer des coups en direction de la gravité).

Variables

— bool(* ia_functions [IA_CLASSIC_SIZE])(game *)

Pointeurs de fonctions pour les différentes IA.

— char * ia_names [IA_CLASSIC_SIZE]

Noms des différentes IA (affichage dans le menu).

4.7.1 Description détaillée

IA pour le mode classique.

Chaque IA implémente une stratégie pour jouer automatiquement. Une fonction d'IA doit jouer un coup dans le jeu et initialiser l'animation correspondante. Si aucun coup n'est possible, elle doit retourner false. En d'autres termes, l'interface graphique (GUI) utilise les fonctions d'IA pour jouer automatiquement en les appelant de manière répétée jusqu'à ce qu'aucun coup ne soit possible.

Définition dans le fichier game_ias.h.

4.7.2 Documentation du type de l'énumération

4.7.2.1 ia_classic

```
enum ia_classic
```

Types d'IA disponibles pour le mode classique.

Valeurs énumérées

IA_RANDOM	IA aléatoire : joue des coups aléatoires.
IA_SAFE	IA prudente : essaie de jouer des coups à l'opposé de la gravité.
IA_AGRESSIVE	IA agressive : essaie de jouer des coups vers la gravité.
IA_CLASSIC_SIZE	Nombre d'IA.

Définition à la ligne 28 du fichier game_ias.h.

4.7.3 Documentation des fonctions

4.7.3.1 ia_random()

```
bool ia_random (
            game * g)
```

Implémentation de l'IA aléatoire (c'est l'IA utilisée dans la démo affichée dans le menu).

Remarques

L'IA doit toujours être capable de jouer un coup (votre implémentation doit s'arrêter lorsqu'aucun coup n'est disponible).

Renvoie

Retourne true si un coup a été joué, false sinon.

Paramètres

g	Le jeu.

4.7.3.2 ia_safe()

```
bool ia_safe (
            game * g)
```

Implémentation de l'IA prudente (l'IA essaie de jouer des coups plus proches du bord opposé à la gravité).

Remarques

L'IA doit toujours être capable de jouer un coup (votre implémentation doit s'arrêter lorsqu'aucun coup n'est disponible).

Renvoie

Retourne true si un coup a été joué, false sinon.

Paramètres

g	Le jeu.

4.7.3.3 ia_agressive()

```
bool ia_agressive (
            game * g)
```

Implémentation de l'IA agressive (l'IA essaie de jouer des coups en direction de la gravité).

Remarques

L'IA doit toujours être capable de jouer un coup (votre implémentation doit s'arrêter lorsqu'aucun coup n'est disponible).

Renvoie

Retourne true si un coup a été joué, false sinon.

Paramètres

g	Le jeu.

4.7.4 Documentation des variables

4.7.4.1 ia_functions

```
bool(* ia_functions[IA_CLASSIC_SIZE])(game *) (
           game * )  [extern]
```

Pointeurs de fonctions pour les différentes IA.

4.7.4.2 ia_names

```
char* ia_names[IA_CLASSIC_SIZE]  [extern]
```

Noms des différentes IA (affichage dans le menu).

4.8 game_ias.h

Aller à la documentation de ce fichier.

```
00001
00012
00013 #ifndef GAME_IAS_H_
00014 #define GAME_IAS_H_
00015
00016 /*   ____ _                _         ____                          ___    _
      */
00017 /*  / ___| | __ _ ___ ___(_) ___   / ___| __ _ _ __ ___   ___  |_ _|  / \   ___
      */
00018 /* | |   | |/ _` / __/ __| |/ __| | |  _ / _` | '_ ` _ \ / _ \  | |  / _ \ / __|
      */
00019 /* | |___| | (_| \__ \__ \ | (__  | |_| | (_| | | | | | |  __/  | | / ___ \\__ \
      */
00020 /*  \____|_|\__,_|___/___/_|\___|  \____|\__,_|_| |_| |_|\___| |___/_/   \_\___/
      */
00021
00022 #include "game.h"
00023
00028 typedef enum
00029 {
00030     IA_RANDOM,
00031     IA_SAFE,
00032     IA_AGRESSIVE,
00033     IA_CLASSIC_SIZE
00034 } ia_classic;
00035
00040 extern bool (*ia_functions[IA_CLASSIC_SIZE])(game *);
00041
00046 extern char *ia_names[IA_CLASSIC_SIZE];
00047
00058 bool ia_random(game *g
```

```
00059 );
00060
00071 bool ia_safe(game *g
00072 );
00073
00084 bool ia_agressive(game *g
00085 );
00086
00087 #endif
```

4.9 Référence du fichier game_play.h

Fonctions appelées par l'interface (GUI) pour gérer les entrées utilisateur. C'est ici que la logique du jeu doit être implémentée.

```
#include "game.h"
#include "game_solver.h"
```

Énumérations

- enum anim_type { ANIM_IDLE , ANIM_MOVE_GOODSWAP , ANIM_MOVE_BADSWAP , ANIM_MOVE_ROLLBACK , ANIM_MOVE_FALL , ANIM_KILL , ANIM_HINT , ANIM_SIZE }

  Les animations disponibles programmées dans l'interface.

- enum end_signal { SIGNAL_END_LEVEL , SIGNAL_END_CLGAME , SIGNAL_END_PUZZLE , SIGNAL_SIZE }

  Utilisé pour signaler la fin d'une partie.

- enum gem_kill { KILL_NOKILL , KILL_SHRINK , KILL_SHATTER , KILL_EXPLODE , KILL_EXPLODE_RING , KILL_SIZE }

  Types d'animations d'élimination disponibles pour les gemmes.

Fonctions

- void game_init_animation (anim_type type)

  Démarre une animation du type spécifié.

- void game_classic_score (void)

Affiche l'écran de score à la fin d'un niveau en mode classique.

— void game_classic_end (void)

Termine une partie classique.

— void game_puzzle_end (void)

Termine une partie puzzle.

— void game_handle_mouse_click (game *g, int i, int j)

Appelée par l'interface lorsqu'un clic souris est effectué sur une case du plateau.

— void game_handle_hint_request (game *g)

Appelée par l'interface lorsqu'une demande d'indice est faite (par ex., clic sur le bouton indice).

— void game_handle_detonation (game *g)

Appelée par l'interface lorsqu'une détonation est demandée par le joueur.

— void game_set_gravity (game *g, direction dir)

Appelée par l'interface lorsqu'un changement de gravité vers une direction donnée est demandé.

— void game_cycle_gravity (game *g)

Appelée par l'interface lorsque la gravité est modifiée dans le sens horaire.

— void game_handle_refill (game *g, int k)

Appelée par l'interface lorsqu'un le joueur veut récupérer un objet dans la file de stockage.

— void game_end_animation (game *g, anim_type type)

Appelée par l'interface lorsqu'une animation se termine.

— direction game_get_cell_move (game *g, int i, int j, int *distance)

Appelée par l'interface uniquement lors des animations MOVE. Elle l'utilise pour récupérer les informations de déplacement des cases à afficher.

— gem_kill game_get_cell_kill (game *g, int i, int j)

Appelée par l'interface uniquement lors d'une animation KILL. Elle l'utilise pour récupérer les informations sur les cases à éliminer et l'animation à utiliser.

— bool game_retrieve_hint (game *g, int *i, int *j)

Appelée par l'interface lorsqu'elle exécute une animation HINT. Sert à récupérer les coordonnées de la case à mettre en évidence.

Variables

— anim_type game_anim_signal

Variable globale utilisée pour signaler à l'interface quelle animation démarrer.

— end_signal game_end_signal

Variable globale utilisée pour indiquer à l'interface quel écran de score afficher.

4.9.1 Description détaillée

Fonctions appelées par l'interface (GUI) pour gérer les entrées utilisateur. C'est ici que la logique du jeu doit être implémentée.

Ce fichier déclare les fonctions que l'interface appelle pour gérer les interactions utilisateur pendant la partie. Vous devez implémenter ces fonctions pour traiter les clics de souris, les demandes d'indice, les détonations, les changements de gravité et les utilisations de la file de stockage. Il vous revient également de gérer les animations :

- Vous pouvez démarrer des animations avec la fonction "game_init_animation" qui est déjà écrite.
- Une fois une animation démarrée, l'interface bloque les entrées utilisateur jusqu'à la fin de l'animation. À ce moment-là, elle appelle la fonction "game_end_animation" que vous devez implémenter pour gérer les étapes suivantes.

Attention

Vous pouvez écrire des fonctions utilitaires supplémentaires si nécessaire, mais vous ne devez PAS modifier les prototypes de fonctions fournis dans ce fichier.

Définition dans le fichier game_play.h.

4.9.2 Documentation du type de l'énumération

4.9.2.1 anim_type

`enum anim_type`

Les animations disponibles programmées dans l'interface.

Valeurs énumérées

ANIM_IDLE	Pas d'animation.

ANIM_MOVE_GOODSWAP	Animation de déplacement pour un échange valide.
ANIM_MOVE_BADSWAP	Animation de déplacement pour un échange invalide (qui devra être annulé après l'animation).
ANIM_MOVE_ROLLBACK	Animation de déplacement pour annuler un échange invalide.
ANIM_MOVE_FALL	Animation de chute des gemmes.
ANIM_KILL	Animation d'élimination des gemmes.
ANIM_HINT	Animation d'indice.
ANIM_SIZE	Nombre de types d'animation.

Définition à la ligne 41 du fichier game_play.h.

4.9.2.2 end_signal

`enum end_signal`

Utilisé pour signaler la fin d'une partie.

Attention

DÉJÀ ÉCRIT. Ne pas modifier.

Valeurs énumérées

SIGNAL_END_LEVEL	Fin de niveau en mode classique.
SIGNAL_END_CLGAME	Fin de partie en mode classique.
SIGNAL_END_PUZZLE	Fin de partie puzzle.
SIGNAL_SIZE	Nombre de signaux.

Définition à la ligne 94 du fichier game_play.h.

4.9.2.3 gem_kill

`enum gem_kill`

Types d'animations d'élimination disponibles pour les gemmes.

Ces types indiquent comment une gemme doit être animée lorsqu'elle est retirée du plateau. Cette information est utilisée par l'interface pour afficher l'animation appropriée.

Remarques

La différence entre KILL_SHRINK, KILL_SHATTER, KILL_EXPLODE et KILL_EXPLODE↩ _RING est uniquement visuelle.

Valeurs énumérées

KILL_NOKILL	Gemme non éliminée (pas d'animation).
KILL_SHRINK	Gemme qui rétrécit (animation standard).
KILL_SHATTER	Gemme qui éclate (pour les gemmes éliminées par un hypercube ou une gemme étoilée).
KILL_EXPLODE	Gemme qui explose (pour les gemmes éliminées par une gemme enflammée ou une bombe).
KILL_EXPLODE_RING	Gemme qui explose en anneau (pour les gemmes enflammées et les bombes elles-mêmes).
KILL_SIZE	Taille de l'énumération gem_kill.

Définition à la ligne 328 du fichier game_play.h.

4.9.3 Documentation des fonctions

4.9.3.1 game_init_animation()

```
void game_init_animation (
           anim_type type)
```

Démarre une animation du type spécifié.

Appelez cette fonction lorsque vous voulez démarrer une animation. Elle bloque les entrées du jeu jusqu'à ce que l'animation soit terminée. À ce moment-là, la fonction game_end_animation (que vous devez écrire) est appelée. Cela vous permet de gérer la fin de l'animation et la transition vers l'état de jeu suivant.

Remarques

> Puisque l'implémentation du plateau vous revient, l'interface ne peut pas récupérer directement dans la structure de jeu les informations nécessaires pour afficher l'animation. Elle le fait en appelant les fonctions déclarées à la fin de ce fichier. Vous devez donc écrire ces fonctions pour que les animations s'affichent correctement.

Attention

> Cette fonction est DÉJÀ ÉCRITE. Vous ne devez pas la modifier.

Paramètres

type	Type d'animation à démarrer.

4.9.3.2 game_classic_score()

```
void game_classic_score (
          void )
```

Affiche l'écran de score à la fin d'un niveau en mode classique.

Appelez cette fonction lorsque le joueur a terminé un niveau d'un jeu classique. Elle affiche l'écran de score. Le niveau suivant démarre lorsque l'utilisateur clique pour continuer.

Attention

Cette fonction est DÉJÀ ÉCRITE. Vous ne devez pas la modifier.

4.9.3.3 game_classic_end()

```
void game_classic_end (
          void )
```

Termine une partie classique.

Appelez cette fonction lorsqu'il n'y a plus de coups disponibles en mode classique. Elle met fin à la partie et affiche l'écran de score final.

Attention

Cette fonction est DÉJÀ ÉCRITE. Vous ne devez pas la modifier.

4.9.3.4 game_puzzle_end()

```
void game_puzzle_end (
          void )
```

Termine une partie puzzle.

Appelez cette fonction lorsqu'il ne reste plus de gemmes sur le plateau en mode puzzle. Elle met fin à la partie et affiche l'écran de victoire.

Attention

Cette fonction est DÉJÀ ÉCRITE. Vous ne devez pas la modifier.

4.9.3.5 game_handle_mouse_click()

```
void game_handle_mouse_click (
            game * g,
            int i,
            int j)
```

Appelée par l'interface lorsqu'un clic souris est effectué sur une case du plateau.

Cette fonction est appelée lorsque l'utilisateur clique sur une case du plateau. Si les coordonnées sont valides, la fonction doit effectuer les étapes suivantes :

- Si aucune case n'est actuellement sélectionnée, sélectionner la case cliquée sans démarrer d'animation.
- Si une case est déjà sélectionnée mais que c'est la même que la case cliquée, la désélectionner sans démarrer d'animation.
- Si une case différente est déjà sélectionnée et que la case cliquée n'est PAS adjacente, désélectionner la case précédemment sélectionnée et sélectionner la case cliquée sans démarrer d'animation.
- Si une case différente est déjà sélectionnée et que la case cliquée EST adjacente, tenter d'échanger la case sélectionnée avec la case cliquée. Si l'échange est valide (c.-à-d. crée au moins une zone destructible), démarrer une animation GOODSWAP. S'il est invalide (c.-à-d. ne crée aucune zone destructible), démarrer une animation BADSWAP.

Attention

Lorsqu'une animation est démarrée, l'interface bloque les entrées jusqu'à la fin de l'animation. À ce moment-là, elle appelle la fonction "game_end_animation" que vous devez implémenter pour gérer la suite. Par exemple, après une animation GOODSWAP, vous devrez démarrer une animation KILL car des zones destructibles sont à supprimer. À l'inverse, après une animation BADSWAP, vous devrez annuler l'échange avec une animation ROLLBACK.

Paramètres

g	Pointeur vers la structure du jeu.
i	Coordonnée horizontale de la case dans le plateau (de gauche à droite).
j	Coordonnée verticale de la case dans le plateau (de haut en bas).

4.9.3.6 game_handle_hint_request()

```
void game_handle_hint_request (
            game * g)
```

Appelée par l'interface lorsqu'une demande d'indice est faite (par ex., clic sur le bouton indice).

Cette fonction vérifie si un coup est disponible sur le plateau. Le cas échéant, elle propose un indice en sélectionnant une case qui peut être échangée pour créer un alignement et démarre l'animation HINT pour la mettre en évidence.

Paramètres

g	Pointeur vers la structure du jeu.

4.9.3.7 game_handle_detonation()

```
void game_handle_detonation (
            game * g)
```

Appelée par l'interface lorsqu'une détonation est demandée par le joueur.

Cette fonction n'est appelée que si le jeu n'est PAS en mode nodelay (c.-à-d. les gemmes ne sont pas supprimées automatiquement après chaque échange). Elle gère la suppression des gemmes : vérifie s'il y a des gemmes à supprimer, met à jour l'état du jeu, puis démarre une animation KILL.

Paramètres

g	Pointeur vers la structure du jeu.

4.9.3.8 game_set_gravity()

```
void game_set_gravity (
            game * g,
            direction dir)
```

Appelée par l'interface lorsqu'un changement de gravité vers une direction donnée est demandé.

Une fois la gravité changée, la fonction doit vérifier si des gemmes peuvent tomber selon la nouvelle direction de gravité et, le cas échéant, les faire tomber (en particulier, une animation FALL doit être démarrée dans ce cas).

Paramètres

g	Pointeur vers la structure du jeu.
dir	Nouvelle direction de la gravité.

4.9.3.9 game_cycle_gravity()

```
void game_cycle_gravity (
            game * g)
```

Appelée par l'interface lorsque la gravité est modifiée dans le sens horaire.

Une fois la gravité changée, la fonction doit vérifier si des gemmes peuvent tomber selon la nouvelle direction de gravité et, le cas échéant, les faire tomber (en particulier, une animation FALL doit être démarrée dans ce cas).

Paramètres

g	Pointeur vers la structure du jeu.

4.9.3.10 game_handle_refill()

```
void game_handle_refill (
            game * g,
            int k)
```

Appelée par l'interface lorsqu'un le joueur veut récupérer un objet dans la file de stockage.

Cette fonction remplit le plateau depuis la file de stockage (si cette file n'est pas vide). Elle fait tomber l'objet récupéré depuis le côté opposé à la gravité à partir de l'indice spécifié en entrée.

Cette fonction ne peut être utilisée qu'en mode puzzle lorsque la file de stockage est activée. Après avoir placé la gemme sur le plateau, la fonction doit démarrer une animation FALL pour faire tomber l'objet jusqu'à sa position.

Remarques

> Si la demande ne peut pas être satisfaite (par exemple parce que la file de stockage est vide ou qu'il n'y a pas de place disponible sur le plateau pour placer la gemme), la fonction doit retourner sans rien faire (en particulier, aucune animation n'est démarrée).

Paramètres

k	Indice de la file de stockage à utiliser pour le remplissage.

4.9.3.11 game_end_animation()

```
void game_end_animation (
            game * g,
            anim_type type)
```

Appelée par l'interface lorsqu'une animation se termine.

Cette fonction est appelée lorsqu'une animation démarrée (par vous) avec game_init_animation se termine. Elle fournit le type d'animation qui vient de se terminer afin que vous puissiez gérer la suite du jeu. La suite dépend du type d'animation et de l'état du jeu. Deux exemples :

Quand une animation FALL se termine, vous devez vérifier si des gemmes peuvent être éliminées,

- si oui, mettez à jour l'état du jeu en conséquence et démarrez une animation KILL pour supprimer ces gemmes ;
- sinon, revenez à l'animation IDLE (qui permet de nouvelles entrées utilisateur).

Quand une animation KILL se termine, vous devez vérifier si des gemmes peuvent tomber selon la gravité,

- si oui, mettez à jour l'état du jeu en conséquence et démarrez une animation FALL pour les faire tomber ;
- sinon, revenez à l'animation IDLE (qui permet de nouvelles entrées utilisateur).

Remarques

C'est également ici que vous devez mettre fin à une partie :

- en mode classique, si un niveau a été terminé, appelez la fonction game_classic_score ;
- en mode classique, si le joueur n'a plus de coups (le joueur a perdu), appelez game_↩ classic_end ;
- en mode puzzle, si le plateau est vide (le joueur a gagné), appelez game_puzzle_end.

Paramètres

g	Pointeur vers la structure du jeu.
type	Type de l'animation qui vient de se terminer.

4.9.3.12 game_get_cell_move()

```
direction game_get_cell_move (
            game * g,
            int i,
            int j,
            int * distance)
```

Appelée par l'interface uniquement lors des animations MOVE. Elle l'utilise pour récupérer les informations de déplacement des cases à afficher.

Lors du premier appel (au début de l'animation), chaque élément doit déjà être à sa position finale (c.-à-d. après l'application du mouvement sur le plateau). Pour une case donnée, l'interface appelle cette fonction afin de savoir si le contenu de la case doit être déplacé depuis une autre position vers sa position actuelle et, le cas échéant, la direction et la distance du déplacement.

Par exemple, si la fonction renvoie la direction LEFT et une distance 3 pour une case donnée, cela signifie que durant l'animation MOVE, le contenu démarrera 3 cases à DROITE de sa position actuelle et se déplacera vers la GAUCHE pour atteindre sa position finale.

Remarques

Les quatre animations de déplacement sont gérées de manière identique par l'interface.

Renvoie

La direction du déplacement. La distance parcourue est affectée au pointeur fourni.

Paramètres

g	Pointeur vers la structure du jeu.
i	Coordonnée horizontale de la case dans le plateau (de gauche à droite).

j	Coordonnée verticale de la case dans le plateau (de haut en bas).
distance	Pointeur pour stocker la distance (peut être NULL si non nécessaire).

4.9.3.13 game_get_cell_kill()

```
gem_kill game_get_cell_kill (
            game * g,
            int i,
            int j)
```

Appelée par l'interface uniquement lors d'une animation KILL. Elle l'utilise pour récupérer les informations sur les cases à éliminer et l'animation à utiliser.

Lors de l'appel, les éléments à éliminer doivent encore être sur le plateau (au démarrage d'une animation KILL, les éléments ne sont pas encore retirés ; ils sont seulement marqués, car l'interface doit connaître les types de gemmes à afficher). Pour chaque case, l'interface appelle cette fonction pour savoir si la case est marquée pour élimination et, le cas échéant, quelle animation utiliser.

Renvoie

KILL_NOKILL si la case n'est pas marquée pour élimination ; sinon, l'animation d'élimination à afficher.

Paramètres

g	Pointeur vers la structure du jeu.
i	Coordonnée horizontale de la case dans le plateau (de gauche à droite).
j	Coordonnée verticale de la case dans le plateau (de haut en bas).

4.9.3.14 game_retrieve_hint()

```
bool game_retrieve_hint (
            game * g,
            int * i,
            int * j)
```

Appelée par l'interface lorsqu'elle exécute une animation HINT. Sert à récupérer les coordonnées de la case à mettre en évidence.

Renvoie

true si une case est effectivement mise en évidence, false sinon (ne devrait pas se produire puisqu'une animation HINT a été lancée).

Paramètres

g	Pointeur vers la structure du jeu.
i	Pointeur pour retourner la coordonnée horizontale de la case surlignée (de gauche à droite).
j	Pointeur pour retourner la coordonnée verticale de la case surlignée (de haut en bas).

4.9.4 Documentation des variables

4.9.4.1 game_anim_signal

```
anim_type game_anim_signal [extern]
```

Variable globale utilisée pour signaler à l'interface quelle animation démarrer.

Attention

Cette variable est DÉJÀ ÉCRITE. Vous ne devez pas la modifier.

4.9.4.2 game_end_signal

```
end_signal game_end_signal [extern]
```

Variable globale utilisée pour indiquer à l'interface quel écran de score afficher.

Attention

Cette variable est DÉJÀ ÉCRITE. Vous ne devez pas la modifier.

4.10 game_play.h

Aller à la documentation de ce fichier.

```
00001
00019
00020 #ifndef GAME_PLAY_H_
00021 #define GAME_PLAY_H_
00022
00023 /*   ____                                _              */
00024 /*  / ___| __ _ _ __ ___   ___ _ __ | | __ _ _   _  */
00025 /* | |  _ / _‘ | ’_ ` _ \ / _ \ ’_ \| |/ _‘ | | | | */
00026 /* | |_| | (_| | | | | | |  __/ |_) | | (_| | |_| | */
00027 /*  \____|\__,_|_| |_| |_|\___| .__/|_|\__,_|\__, | */
00028 /*                            |_|            |___/  */
00029
00030 #include "game.h"
00031 #include "game_solver.h"
00032
```

```
00033
      /*****************************************************************************************************
00034 /*+ Fonction qui démarre une animation (ou revient au mode IDLE). Déjà écrite :
      appelez-la mais NE PAS MODIFIER +*/
00035
      /*****************************************************************************************************
00036
00041 typedef enum
00042 {
00043     ANIM_IDLE,
00044     ANIM_MOVE_GOODSWAP,
00045     ANIM_MOVE_BADSWAP,
00046     ANIM_MOVE_ROLLBACK,
00047     ANIM_MOVE_FALL,
00048     ANIM_KILL,
00049     ANIM_HINT,
00050     ANIM_SIZE
00051 } anim_type;
00052
00060 extern anim_type game_anim_signal;
00061
00079 void game_init_animation(anim_type type
00080 );
00081
00082
      /*****************************************************************************************************
00083 /*+ Fonctions qui terminent la partie. Déjà écrites : appelez-les quand la
      partie est terminée. NE PAS+*/
00084 /*+ MODIFIER
      +*/
00085
      /*****************************************************************************************************
00086
00094 typedef enum
00095 {
00096     SIGNAL_END_LEVEL,
00097     SIGNAL_END_CLGAME,
00098     SIGNAL_END_PUZZLE,
00099     SIGNAL_SIZE
00100 } end_signal;
00101
00109 extern end_signal game_end_signal;
00110
00122 void game_classic_score(void);
00123
00135 void game_classic_end(void);
00136
00148 void game_puzzle_end(void);
00149
00150
      *****************************************************************************************/
00151 /*+ Fonctions appelées par l'interface pour gérer les entrées et les
      modifications  +*/
00152 /*+ de l'état du jeu. Vous devez implémenter ces fonctions (elles démarrent des
      +*/
00153 /*+ animations).
      +*/
00154
```

```
    /*****************************************************************************************/
00155
00179 void game_handle_mouse_click(game *g,
00180                              int i,
00181                              int j
00182 );
00183
00192 void game_handle_hint_request(game *g
00193 );
00194
00204 void game_handle_detonation(game *g
00205 );
00206
00215 void game_set_gravity(game *g,
00216                       direction dir
00217 );
00218
00227 void game_cycle_gravity(game *g
00228 );
00229
00250 void game_handle_refill(game *g, //< Pointeur vers la structure du jeu.
00251                         int k
00252 );
00253
00279 void game_end_animation(game *g,
00280                         anim_type type
00281 );
00282
00283
    /*****************************************************************************************/
00284 /*+ Fonctions utilisées par l'interface pour récupérer des informations
    lorsqu'elle  +*/
00285 /*+ exécute une animation. Vous devez implémenter ces fonctions pour que les
    +*/
00286 /*+ animations s'affichent correctement.
    +*/
00287
    /*****************************************************************************************/
00288
00311 direction game_get_cell_move(game *g,
00312                              int i,
00313                              int j,
00314                              int *distance
00315 );
00316
00328 typedef enum
00329 {
00330     KILL_NOKILL,
00331     KILL_SHRINK,
00332     KILL_SHATTER,
00333     KILL_EXPLODE,
00334     KILL_EXPLODE_RING,
00335     KILL_SIZE
00336 } gem_kill;
00337
00352 gem_kill game_get_cell_kill(game *g,
00353                             int i,
00354                             int j
```

```
00355 );
00356
00365 bool game_retrieve_hint(game *g,
00366                          int *i,
00367                          int *j
00368 );
00369
00370 #endif // GAME_PLAY_H_
```

4.11 Référence du fichier game_save.h

Sauvegarde et chargement des états de jeu.

```
#include "game.h"
#include "game_scores.h"
```

Énumérations

— enum save_keywords {
SK_WIDTH , SK_HEIGHT , SK_BOARD , SK_GRAVITY ,
SK_REFILL_QUEUE , SK_PUZZLE , SK_LINES , SK_UPGRADES ,
SK_NODELAY , SK_EASY , SK_WRAP , SK_BOMBS ,
SK_REFILL , SK_GRAVITY_CONTROL , SK_LEVEL , SK_LEVEL_SCORE ,
SK_TARGET_SCORE , SK_SCALE_SCORE , SK_SCORE_ARRAY , SK_REFILL_COUNT ,
SK_SIZE }

Mots-clés représentant les différents paramètres pouvant être sauvegardés/chargés dans un fichier d'état de jeu.

Fonctions

— bool save_game (game *g, const char *filepath)

Sauvegarde l'état de jeu courant dans un fichier.

— game * load_game (const char *filepath)

Charge un état de jeu depuis un fichier.

Variables

— const char * save_keyword_strings [SK_SIZE]

Chaînes correspondant aux mots-clés utilisés dans le fichier de sauvegarde.

4.11.1 Description détaillée

Sauvegarde et chargement des états de jeu.

game_save.h

Fonctions utilisées pour sauvegarder et charger des états de jeu. Vous devez respecter le format spécifié pour les fichiers de sauvegarde afin d'assurer la compatibilité (voir la documentation du projet pour les détails).

Définition dans le fichier game_save.h.

4.11.2 Documentation du type de l'énumération

4.11.2.1 save_keywords

```
enum save_keywords
```

Mots-clés représentant les différents paramètres pouvant être sauvegardés/chargés dans un fichier d'état de jeu.

Remarques

Rappel : tous les paramètres ne sont pas obligatoires dans un fichier de sauvegarde. Les seuls paramètres obligatoires sont les suivants (les autres peuvent être initialisés à des valeurs par défaut s'ils sont absents) :

- La largeur et la hauteur du plateau (SK_WIDTH et SK_HEIGHT).
- L'état du plateau (SK_BOARD).
- Le mode de jeu (SK_PUZZLE).

Valeurs énumérées

SK_WIDTH	Largeur du plateau de jeu.
SK_HEIGHT	Hauteur du plateau de jeu.
SK_BOARD	État du plateau de jeu.
SK_GRAVITY	Direction de la gravité.
SK_REFILL_QUEUE	File de remplissage elle-même (pour les modes de remplissage 0 ou 1).
SK_PUZZLE	Mode de jeu (puzzle ou classique).
SK_LINES	Spécification des zones éliminables.
SK_UPGRADES	Améliorations activées dans le jeu.
SK_NODELAY	Mode sans délai pour l'élimination des gemmes.
SK_EASY	Mode facile du jeu.

SK_WRAP	Mode wrap des gemmes pour le jeu.
SK_BOMBS	Mode bombes du jeu.
SK_REFILL	Mode de remplissage du jeu.
SK_GRAVITY_CONTROL	Mode de contrôle de la gravité.
SK_LEVEL	Niveau actuel du jeu.
SK_LEVEL_SCORE	Score du niveau en cours.
SK_TARGET_SCORE	Score cible pour le niveau en cours.
SK_SCALE_SCORE	Pourcentage d'échelonnage du score.
SK_SCORE_ARRAY	Tableau des scores pour tous les niveaux.
SK_REFILL_COUNT	Nombre de remplissages déjà utilisés.
SK_SIZE	

Définition à la ligne 39 du fichier game_save.h.

4.11.3 Documentation des fonctions

4.11.3.1 save_game()

```
bool save_game (
            game * g,
            const char * filepath)
```

Sauvegarde l'état de jeu courant dans un fichier.

Renvoie

Retourne true si la sauvegarde a réussi, false sinon.

Paramètres

g	Pointeur vers la structure du jeu.
filepath	Chemin du fichier où l'état de jeu sera sauvegardé.

4.11.3.2 load_game()

```
game * load_game (
            const char * filepath)
```

Charge un état de jeu depuis un fichier.

Renvoie

Retourne un pointeur vers la structure de jeu chargée, ou NULL si le chargement a échoué.

Paramètres

filepath	Chemin du fichier à partir duquel l'état de jeu sera chargé.

4.11.4 Documentation des variables

4.11.4.1 save_keyword_strings

```
const char* save_keyword_strings[SK_SIZE] [extern]
```

Chaînes correspondant aux mots-clés utilisés dans le fichier de sauvegarde.

4.12 game_save.h

Aller à la documentation de ce fichier.

```
00001
00013
00014 #ifndef GAME_SAVE_H
00015 #define GAME_SAVE_H
00016
00017 /*   ____                          ____                          */
00018 /*  / ___| __ _ _ __ ___   ___  / ___|  __ ___   _____  ___  */
00019 /* | |  _ / _` | '_ ` _ \ / _ \ \___ \ / _` \ \ / / _ \/ __| */
00020 /* | |_| | (_| | | | | | |  __/  ___) | (_| |\ V /  __/\__ \ */
00021 /*  \____|\__,_|_| |_| |_|\___| |____/ \__,_| \_/ \___||___/ */
00022
00023 #include "game.h"
00024 #include "game_scores.h"
00025
00039 typedef enum
00040 {
00041     SK_WIDTH,
00042     SK_HEIGHT,
00043     SK_BOARD,
00044     SK_GRAVITY,
00045     SK_REFILL_QUEUE,
00046     SK_PUZZLE,
00047     SK_LINES,
00048     SK_UPGRADES,
00049     SK_NODELAY,
00050     SK_EASY,
00051     SK_WRAP,
00052     SK_BOMBS,
00053     SK_REFILL,
00054     SK_GRAVITY_CONTROL,
00055     SK_LEVEL,
00056     SK_LEVEL_SCORE,
00057     SK_TARGET_SCORE,
00058     SK_SCALE_SCORE,
00059     SK_SCORE_ARRAY,
00060     SK_REFILL_COUNT,
```

```
00061     SK_SIZE
00062 } save_keywords;
00063
00068 extern const char *save_keyword_strings[SK_SIZE];
00069
00077 bool save_game(game *g,
00078                const char *filepath
00079 );
00080
00088 game *load_game(const char *filepath
00089 );
00090
00091 #endif // GAME_SAVE_H
```

4.13 Référence du fichier game_scores.h

Gestion des meilleurs scores en mode classique. Contient les fonctions permettant de charger, sauvegarder et trier les meilleurs scores à partir du fichier de scores.

```
#include "alloc.h"
#include "game.h"
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
```

Structures de données

— struct score_table

Structure permettant de stocker le tableau des scores affiché par l'interface graphique (GUI).

Fonctions

— score_table * load_score_table (const char *filepath, int *player_score)

Appelée par l'interface graphique (GUI) pour charger le tableau des scores depuis le fichier de scores.

— void save_score_table (score_table *table, const char *filepath, const char *playername)

Appelée par l'interface graphique (GUI) pour sauvegarder une table de scores dans un fichier de scores.

— void delete_score_table (score_table *table)

Appelée par l'interface graphique (GUI) pour supprimer une table de scores et libérer la mémoire allouée.

— void sort_score_table (score_table *table, score_type type)

par l'interface graphique (GUI) pour trier to sort une table de scores en ordre décroissant, sur la base d'un type de score spécifié.

4.13.1 Description détaillée

Gestion des meilleurs scores en mode classique. Contient les fonctions permettant de charger, sauvegarder et trier les meilleurs scores à partir du fichier de scores.

Il est nécessaire de respecter le format spécifié pour le fichier de scores afin d'assurer un chargement et une sauvegarde corrects des scores (voir la documentation du projet pour plus de détails).

Définition dans le fichier game_scores.h.

4.13.2 Documentation des fonctions

4.13.2.1 load_score_table()

```
score_table * load_score_table (
          const char * filepath,
          int * player_score)
```

Appelée par l'interface graphique (GUI) pour charger le tableau des scores depuis le fichier de scores.

Cette fonction lit un tableau de scores à partir du fichier spécifié. Si le fichier n'existe pas, elle crée un tableau de scores vide. Si player_score est fourni, il est ajouté comme entrée pour le joueur actuel sous le nom « Player ».

Paramètres

filepath	Chemin d'accès sur disque au fichier des scores.
player_score	Pointeur vers le tableau de score du joueur actuel (NULL si utilisé pour charger les meilleurs scores depuis le menu).

4.13.2.2 save_score_table()

```
void save_score_table (
          score_table * table,
          const char * filepath,
          const char * playername)
```

Appelée par l'interface graphique (GUI) pour sauvegarder une table de scores dans un fichier de scores.

Paramètres

table	Pointeur vers la table des scores à sauvegarder.

filepath	Chemin d'accès au fichier de scores.
playername	Nom choisi pour le joueur courant (peut être NULL, quand aucun score du joueur courant n'est chargé dans la table).

4.13.2.3 delete_score_table()

```
void delete_score_table (
            score_table * table)
```

Appelée par l'interface graphique (GUI) pour supprimer une table de scores et libérer la mémoire allouée.

Paramètres

table	Pointeur vers la table des scores à supprimer.

4.13.2.4 sort_score_table()

```
void sort_score_table (
            score_table * table,
            score_type type)
```

par l'interface graphique (GUI) pour trier to sort une table de scores en ordre décroissant, sur la base d'un type de score spécifié.

Paramètres

table	Pointeur vers la table des scores à trier.
type	Type de score selon lequel trier.

4.14 game_scores.h

Aller à la documentation de ce fichier.

```
00001
00014
00015 #ifndef _GAME_SCORES_H_
00016 #define _GAME_SCORES_H_
00017
00018 /*  _   _ _       _       ____                              */
00019 /* | | | (_) __ _| |__   / ___|  ___ ___  _ __ ___  ___   */
00020 /* | |_| | |/ _` | '_ \  \___ \ / __/ _ \| '__/ _ \/ __| */
00021 /* |  _  | | (_| | | | |  ___) | (_| (_) | | |  __/\__ \ */
00022 /* |_| |_|_|\__, |_| |_| |____/ \___\___/|_|  \___||___/ */
```

```
00023 /*          |___/                                          */
00024
00025 #include "alloc.h"
00026 #include "game.h"
00027 #include <stdbool.h>
00028 #include <stdio.h>
00029 #include <stdlib.h>
00030
00054 typedef struct {
00055     char (*names)[64];
00056     int **scores;
00057     int size;
00059     int player_index;
00060     int *permutation;
00062 } score_table;
00063
00075 score_table *load_score_table(
00076     const char *filepath,
00077     int *player_score
00080 );
00081
00087 void save_score_table(
00088     score_table *table,
00089     const char *filepath,
00090     const char *playername
00093 );
00094
00100 void delete_score_table(
00101     score_table *table
00102 );
00103
00109 void sort_score_table(
00110     score_table *table,
00111     score_type type
00112 );
00113
00114 #endif // _GAME_SCORES_H_
```

4.15 Référence du fichier game_solver.h

```
#include "game.h"
```

Structures de données

— struct hash_table

Solveur pour le mode puzzle.

— struct solver_move

Structure récursive représentant une séquence de coups d'une solution de puzzle.

Définitions de type

— typedef struct solver_move solver_move

Structure récursive représentant une séquence de coups d'une solution de puzzle.

Énumérations

— enum solver_move_type { SOLVER_SWAP , SOLVER_GRAVITY , SOLVER_CYCLE , SOLVER_STORAGE }

Types de coups que le solveur peut effectuer (les coups disponibles dépendent des paramètres du jeu).

— enum solver_result_type { SOLVER_TIMEOUT , SOLVER_UNSOLVABLE , SOLVER_SOLVED }

Les trois résultats possibles renvoyés par le solveur.

— enum solver { SOLVER_DFS , SOLVER_BFS , SOLVER_ASTAR }

Résolveurs disponibles.

Fonctions

— hash_table * create_hash_table (int element_size)

Crée une nouvelle table de hachage vide.

— void delete_hash_table (hash_table *ht)

Supprime une table de hachage.

— bool hash_table_insert (hash_table *ht, int *elements, ulong i)

Tente d'insérer un élément dans la table de hachage.

— solver_move * solve_puzzle (game *g, solver method, int bound, solver_result_type *result, int *nb_moves, int *nb_configs)

Appelée par l'interface pour résoudre un puzzle avec le solveur indiqué.

— bool game_implement_solver_move (game *g, solver_move *move)

Implémente dans le jeu un coup fourni par le solveur.

4.15.1 Documentation des définitions de type

4.15.1.1 solver_move

```
typedef struct solver_move solver_move
```

Structure récursive représentant une séquence de coups d'une solution de puzzle.

4.15.2 Documentation du type de l'énumération

4.15.2.1 solver_move_type

```
enum solver_move_type
```

Types de coups que le solveur peut effectuer (les coups disponibles dépendent des paramètres du jeu).

Valeurs énumérées

SOLVER_SWAP	Coup d'échange (entre deux cases adjacentes).
SOLVER_GRAVITY	Coup de gravité (changer la gravité dans une direction quelconque).
SOLVER_CYCLE	Coup de cycle de gravité (rotation horaire).
SOLVER_STORAGE	Coup de stockage (prendre dans la file de remplissage).

Définition à la ligne 100 du fichier game_solver.h.

4.15.2.2 solver_result_type

```
enum solver_result_type
```

Les trois résultats possibles renvoyés par le solveur.

Remarques

Sert à distinguer les deux cas d'échec (dépassement de temps et puzzle insoluble).

Valeurs énumérées

SOLVER_TIMEOUT	Dépassement du temps (itérations max). Certaines configurations n'ont pas été explorées.
SOLVER_UNSOLVABLE	Puzzle insoluble. Toutes les configurations atteignables ont été explorées.

SOLVER_SOLVED	Puzzle soluble. Une solution a été trouvée.

Définition à la ligne 127 du fichier game_solver.h.

4.15.2.3 solver

```
enum solver
```

Résolveurs disponibles.

Valeurs énumérées

SOLVER_DFS	
SOLVER_BFS	
SOLVER_ASTAR	

Définition à la ligne 138 du fichier game_solver.h.

4.15.3 Documentation des fonctions

4.15.3.1 create_hash_table()

```
hash_table * create_hash_table (
            int element_size)
```

Crée une nouvelle table de hachage vide.

Renvoie

La table de hachage vide créée.

Paramètres

element_size	Nombre d'entiers utilisés pour encoder un élément dans le stockage externe.

4.15.3.2 delete_hash_table()

```
void delete_hash_table (
            hash_table * ht)
```

Supprime une table de hachage.

Paramètres

ht	Table de hachage à supprimer.

4.15.3.3 hash_table_insert()

```
bool hash_table_insert (
            hash_table * ht,
            int * elements,
            ulong i)
```

Tente d'insérer un élément dans la table de hachage.

Remarques

L'indice i désigne la position de l'élément dans le tableau de stockage externe. Comme chaque élément est encodé comme une séquence d'entiers de taille fixe (champ element_size), la table de hachage lit la séquence de element_size entiers à partir de la position i * element_size dans ce tableau externe.

Renvoie

true si l'élément a été inséré, false s'il était déjà présent. Dans ce dernier cas, la table n'est pas modifiée.

Paramètres

ht	Table de hachage.
elements	Tableau de stockage externe contenant tous les éléments.
i	Indice de l'élément à insérer dans le stockage externe.

4.15.3.4 solve_puzzle()

```
solver_move * solve_puzzle (
            game * g,
            solver method,
            int bound,
            solver_result_type * result,
            int * nb_moves,
            int * nb_configs)
```

Appelée par l'interface pour résoudre un puzzle avec le solveur indiqué.

Renvoie

Un pointeur vers la liste des coups menant à la solution, ou NULL si aucune solution n'a été trouvée.

Paramètres

g	Pointeur vers la structure du jeu.
method	Méthode de résolution (DFS, BFS, A∗).
bound	Nombre maximal d'itérations avant délai (0 ou moins pour illimité).
result	Pointeur vers une variable où stocker le résultat.
nb_moves	Pointeur vers un entier pour stocker le nombre de coups de la solution.
nb_configs	Pointeur vers un entier pour stocker le nombre de configurations explorées.

4.15.3.5 game_implement_solver_move()

```
bool game_implement_solver_move (
            game * g,
            solver_move * move)
```

Implémente dans le jeu un coup fourni par le solveur.

Cette fonction prend une structure solver_move représentant un coup unique proposé par le solveur et tente de l'appliquer au jeu. Si le coup est valide dans l'état courant du jeu, il est appliqué et une animation doit être démarrée.

Renvoie

true si le coup a été appliqué avec succès, false sinon.

4.16 game_solver.h

Aller à la documentation de ce fichier.

```
00001
00019
00020 #ifndef GAME_SOLVER_H_
00021 #define GAME_SOLVER_H_
00022
00023 /*  ____             _                    */
00024 /* / ___|  ___ | |_   _____ _ __   */
00025 /* \___ \ / _ \| \ \ / / _ \ '__| */
00026 /*  ___) | (_) | |\ V /  __/ |    */
00027 /* |____/ \___/|_| \_/ \___|_|    */
00028
00029 #include "game.h"
00030
00031 /*************************************/
00032 /*+ Table de hachage (déjà écrite)  +*/
00033 /*************************************/
00034
00049 typedef struct
00050 {
00051     ulong hash_capacity;
00052     ulong hash_size;
```

```
00053     ulong *hash_table;
00054     int element_size;
00055 } hash_table;
00056
00064 hash_table *create_hash_table(
00065     int element_size
00066 );
00067
00072 void delete_hash_table(hash_table *ht
00073 );
00074
00087 bool hash_table_insert(hash_table *ht,
00088                        int *elements,
00089                        ulong i
00090 );
00091
00092 /**********************/
00093 /*+ Fonctions solveur +*/
00094 /**********************/
00095
00100 typedef enum
00101 {
00102     SOLVER_SWAP,
00103     SOLVER_GRAVITY,
00104     SOLVER_CYCLE,
00105     SOLVER_STORAGE
00106 } solver_move_type;
00107
00112 typedef struct solver_move
00113 {
00114     solver_move_type type;
00115     int index;
00116     direction dir;
00117     struct solver_move *next;
00118 } solver_move;
00119
00127 typedef enum
00128 {
00129     SOLVER_TIMEOUT,
00130     SOLVER_UNSOLVABLE,
00131     SOLVER_SOLVED,
00132 } solver_result_type;
00133
00138 typedef enum
00139 {
00140     SOLVER_DFS,
00141     SOLVER_BFS,
00142     SOLVER_ASTAR
00143 } solver;
00144
00152 solver_move *solve_puzzle(game *g,
00153                           solver method,
00154                           int bound,
00155                           solver_result_type *result,
00156                           int *nb_moves,
00157                           int *nb_configs
00158 );
00159
```

```
00172 bool game_implement_solver_move(game *g, solver_move *move);
00173
00174 #endif // GAME_SOLVER_H_
```

4.17 Référence du fichier project_parameters.h

Paramètres.

Macros

— #define WINDOW_SCREENWIDTH 1820

Dimension horizontale de l'application en pixels (valeur par défaut = 980).

— #define WINDOW_SCREENHEIGHT 980

Dimension verticale de l'application en pixels (valeur par défaut = 980).

— #define GAME_WIDTH 8

Largeur par défaut du plateau de jeu en nombre de cases (valeur par défaut = 8).

— #define GAME_HEIGHT 8

Hauteur par défaut du plateau de jeu en nombre de cases (valeur par défaut = 8).

— #define GAME_LINES true

Mode d'élimination des gemmes (true pour les lignes ou false pour les SCC) (valeur par défaut = true).

— #define GAME_NODELAY true

Paramètre d'activation du mode nodelay (valeur par défaut = true). Spécifique au mode classique (toujours actif en mode puzzle).

— #define GAME_EASY false

Paramètre d'activation du mode facile (valeur par défaut = faux). Spécifique au mode classique.

— #define GAME_WRAP false

Paramètre d'activation des gemmes de gravité (valeur par défaut = false). Spécifique au mode classique.

— #define GAME_BOMBS false

Paramètre d'activation des bombes (valeur par défaut = faux). Spécifique au mode classique.

— #define GAME_UPGRADES true

Paramètre d'activation pour les mises à jour des gemmes (valeur par défaut = true). Pertinent à la fois en mode classique et puzzle.

— #define GAME_REFILL_MODE 1

Mode de remplissage en mode puzzle (0: stocke les gemmes éliminées, 1: file prédéterminée de rochers et de bombes, 2: Pas de remplissage) (par défaut = 2).

— #define GAME_BALANCED true

Paramètre d'activation pour la génération équilibrée en mode puzzle (par défaut = true).

— #define GAME_STORAGE_INITIAL 2

Taille initiale de la file d'attente de stockage en mode puzzle (uniquement pertinent si refill_mode vaut 1) (valeur par défaut = 2).

— #define GAME_CLEARSCORE 10000

Score initial pour terminer le premier niveau en mode classique (par défaut = 10000).

— #define GAME_CLEARMULTI 50

Pourcentage d'augmentation du score pour passer au niveau suivant en mode classique (valeur par défaut = 50 %). Multiplicateur de score appliqué au score pour progresser au niveau suivant, après chaque niveau terminé en mode classique.

— #define TEST_WIDTH 8

Largeur du plateau de jeu en nombre de cellules (valeur par défaut = 8).

— #define TEST_HEIGHT 8

Hauteur du plateau de jeu en nombre de cellules (valeur par défaut = 8).

— #define TEST_PUZZLE false

Paramètre d'activation du mode puzzle (valeur par défaut = false).

— #define TEST_LINES true

Mode d'élimination des gemmes (true pour les lignes, false pour les SCC) (valeur par défaut = true). Pertinent à la fois en mode classique et en mode puzzle.

— #define TEST_NODELAY true

Paramètre d'activation du mode nodelay (valeur par défaut = true). Pertinent uniquement en mode classique.

— #define TEST_EASY false

Paramètre d'activation du mode easy (valeur par défaut = false). Pertinent uniquement en mode classique.

— #define TEST_WRAP false

Paramètre d'activation des gemmes de gravité (valeur par défaut = false). Pertinent uniquement en mode classique.

— #define TEST_BOMBS false

Paramètre d'activation pour les bombes (valeur par défaut = false). Pertinent uniquement en mode classique.

— #define TEST_UPGRADES true

Paramètre d'activation pour les améliorations (valeur par défaut = true). Pertinent à la fois en mode classique et en mode puzzle.

— #define TEST_REFILL_MODE 1

Mode de recharge de la file en mode puzzle (0: stocke les gemmes éliminées, 1: file prédéterminée de rochers et de bombes, 2: Pas de recharge) (valeur par défaut = 1).

— #define TEST_STORAGE_INITIAL 2

Taille initiale de la file de stockage en mode puzzle (seulement pertinent si refill_mode vaut 1) (valeur par défaut = 2).

— #define TEST_BALANCED true

Paramètre d'activation pour la génération équilibrée en mode puzzle (par défaut = true).

— #define SOLVER_MAX_CONFIGS 4000000

Nombre maximal de configurations explorées par le solveur avant d'abandonner (valeur par défaut = 4 000 000). Fixé à -1 pour ne pas avoir de limite.

4.17.1 Description détaillée

Paramètres.

Valeurs par défaut des paramètres des jeux.

Définition dans le fichier project_parameters.h.

4.17.2 Documentation des macros

4.17.2.1 WINDOW_SCREENWIDTH

```
#define WINDOW_SCREENWIDTH 1820
```

Dimension horizontale de l'application en pixels (valeur par défaut = 980).

Définition à la ligne 25 du fichier project_parameters.h.

4.17.2.2 WINDOW_SCREENHEIGHT

```
#define WINDOW_SCREENHEIGHT 980
```

Dimension verticale de l'application en pixels (valeur par défaut = 980).

Définition à la ligne 31 du fichier project_parameters.h.

4.17.2.3 GAME_WIDTH

```
#define GAME_WIDTH 8
```

Largeur par défaut du plateau de jeu en nombre de cases (valeur par défaut = 8).

Définition à la ligne 42 du fichier project_parameters.h.

4.17.2.4 GAME_HEIGHT

```
#define GAME_HEIGHT 8
```

Hauteur par défaut du plateau de jeu en nombre de cases (valeur par défaut = 8).

Définition à la ligne 49 du fichier project_parameters.h.

4.17.2.5 GAME_LINES

```
#define GAME_LINES true
```

Mode d'élimination des gemmes (true pour les lignes ou false pour les SCC) (valeur par défaut = true).

Définition à la ligne 56 du fichier project_parameters.h.

4.17.2.6 GAME_NODELAY

```
#define GAME_NODELAY true
```

Paramètre d'activation du mode nodelay (valeur par défaut = true). Spécifique au mode classique (toujours actif en mode puzzle).

Définition à la ligne 63 du fichier project_parameters.h.

4.17.2.7 GAME_EASY

```
#define GAME_EASY false
```

Paramètre d'activation du mode facile (valeur par défaut = faux). Spécifique au mode classique.

Définition à la ligne 70 du fichier project_parameters.h.

4.17.2.8 GAME_WRAP

```
#define GAME_WRAP false
```

Paramètre d'activation des gemmes de gravité (valeur par défaut = false). Spécifique au mode classique.

Définition à la ligne 76 du fichier project_parameters.h.

4.17.2.9 GAME_BOMBS

```
#define GAME_BOMBS false
```

Paramètre d'activation des bombes (valeur par défaut = faux). Spécifique au mode classique.

Définition à la ligne 82 du fichier project_parameters.h.

4.17.2.10 GAME_UPGRADES

```
#define GAME_UPGRADES true
```

Paramètre d'activation pour les mises à jour des gemmes (valeur par défaut = true). Pertinent à la fois en mode classique et puzzle.

Définition à la ligne 89 du fichier project_parameters.h.

4.17.2.11 GAME_REFILL_MODE

```
#define GAME_REFILL_MODE 1
```

Mode de remplissage en mode puzzle (0: stocke les gemmes éliminées, 1: file prédéterminée de rochers et de bombes, 2: Pas de remplissage) (par défaut = 2).

Définition à la ligne 97 du fichier project_parameters.h.

4.17.2.12 GAME_BALANCED

```
#define GAME_BALANCED true
```

Paramètre d'activation pour la génération équilibrée en mode puzzle (par défaut = true).

Définition à la ligne 104 du fichier project_parameters.h.

4.17.2.13 GAME_STORAGE_INITIAL

```
#define GAME_STORAGE_INITIAL 2
```

Taille initiale de la file d'attente de stockage en mode puzzle (uniquement pertinent si refill_mode vaut 1) (valeur par défaut = 2).

Définition à la ligne 111 du fichier project_parameters.h.

4.17.2.14 GAME_CLEARSCORE

```
#define GAME_CLEARSCORE 10000
```

Score initial pour terminer le premier niveau en mode classique (par défaut = 10000).

Définition à la ligne 117 du fichier project_parameters.h.

4.17.2.15 GAME_CLEARMULTI

```
#define GAME_CLEARMULTI 50
```

Pourcentage d'augmentation du score pour passer au niveau suivant en mode classique (valeur par défaut = 50 %). Multiplicateur de score appliqué au score pour progresser au niveau suivant, après chaque niveau terminé en mode classique.

Définition à la ligne 125 du fichier project_parameters.h.

4.17.2.16 TEST_WIDTH

```
#define TEST_WIDTH 8
```

Largeur du plateau de jeu en nombre de cellules (valeur par défaut = 8).

Définition à la ligne 135 du fichier project_parameters.h.

4.17.2.17 TEST_HEIGHT

```
#define TEST_HEIGHT 8
```

Hauteur du plateau de jeu en nombre de cellules (valeur par défaut = 8).

Définition à la ligne 141 du fichier project_parameters.h.

4.17.2.18 TEST_PUZZLE

```
#define TEST_PUZZLE false
```

Paramètre d'activation du mode puzzle (valeur par défaut = false).

Définition à la ligne 146 du fichier project_parameters.h.

4.17.2.19 TEST_LINES

```
#define TEST_LINES true
```

Mode d'élimination des gemmes (true pour les lignes, false pour les SCC) (valeur par défaut = true). Pertinent à la fois en mode classique et en mode puzzle.

Définition à la ligne 153 du fichier project_parameters.h.

4.17.2.20 TEST_NODELAY

```
#define TEST_NODELAY true
```

Paramètre d'activation du mode nodelay (valeur par défaut = true). Pertinent uniquement en mode classique.

Définition à la ligne 159 du fichier project_parameters.h.

4.17.2.21 TEST_EASY

```
#define TEST_EASY false
```

Paramètre d'activation du mode easy (valeur par défaut = false). Pertinent uniquement en mode classique.

Définition à la ligne 165 du fichier project_parameters.h.

4.17.2.22 TEST_WRAP

```
#define TEST_WRAP false
```

Paramètre d'activation des gemmes de gravité (valeur par défaut = false). Pertinent uniquement en mode classique.

Définition à la ligne 172 du fichier project_parameters.h.

4.17.2.23 TEST_BOMBS

```
#define TEST_BOMBS false
```

Paramètre d'activation pour les bombes (valeur par défaut = false). Pertinent uniquement en mode classique.

Définition à la ligne 178 du fichier project_parameters.h.

4.17.2.24 TEST_UPGRADES

```
#define TEST_UPGRADES true
```

Paramètre d'activation pour les améliorations (valeur par défaut = true). Pertinent à la fois en mode classique et en mode puzzle.

Définition à la ligne 184 du fichier project_parameters.h.

4.17.2.25 TEST_REFILL_MODE

```
#define TEST_REFILL_MODE 1
```

Mode de recharge de la file en mode puzzle (0: stocke les gemmes éliminées, 1: file prédéterminée de rochers et de bombes, 2: Pas de recharge) (valeur par défaut = 1).

Définition à la ligne 191 du fichier project_parameters.h.

4.17.2.26 TEST_STORAGE_INITIAL

```
#define TEST_STORAGE_INITIAL 2
```

Taille initiale de la file de stockage en mode puzzle (seulement pertinent si refill_mode vaut 1) (valeur par défaut = 2).

Définition à la ligne 197 du fichier project_parameters.h.

4.17.2.27 TEST_BALANCED

```
#define TEST_BALANCED true
```

Paramètre d'activation pour la génération équilibrée en mode puzzle (par défaut = true).

Définition à la ligne 203 du fichier project_parameters.h.

4.17.2.28 SOLVER_MAX_CONFIGS

```
#define SOLVER_MAX_CONFIGS  4000000
```

Nombre maximal de configurations explorées par le solveur avant d'abandonner (valeur par défaut = 4 000 000). Fixé à -1 pour ne pas avoir de limite.

Nombre maximal de configurations explorées par le solveur avant d'abandonner.

Définition à la ligne 215 du fichier project_parameters.h.

4.18 project_parameters.h

Aller à la documentation de ce fichier.

```
00001
00007
00008 #ifndef _PROJECT_PARAMETERS_H_
00009 #define _PROJECT_PARAMETERS_H_
00010
00011 /*  ____                                    _                 */
00012 /* |  _ \ __ _ _ __ __ _ _ __ ___   ___| |_ ___ _ __ ___  */
00013 /* | |_) / _` | '__/ _` | '_ ` _ \ / _ \ __/ _ \ '__/ __| */
00014 /* |  __/ (_| | | | (_| | | | | | |  __/ ||  __/ |  \__ \ */
00015 /* |_|   \__,_|_|  \__,_|_| |_| |_|\___|\__\___|_|  |___/ */
00016
00017 /************************/
00018 /*+ Window dimensions +*/
00019 /************************/
00020
00025 #define WINDOW_SCREENWIDTH   1820
00026
00031 #define WINDOW_SCREENHEIGHT  980
00032
00033 /************************/
00034 /*+ Default parameters +*/
00035 /************************/
00036
00042 #define GAME_WIDTH           8
00043
00049 #define GAME_HEIGHT          8
00050
00056 #define GAME_LINES           true
```

```
00057
00063 #define GAME_NODELAY         true
00064
00070 #define GAME_EASY            false
00071
00076 #define GAME_WRAP            false
00077
00082 #define GAME_BOMBS           false
00083
00089 #define GAME_UPGRADES        true
00090
00097 #define GAME_REFILL_MODE     1
00098
00104 #define GAME_BALANCED        true
00105
00111 #define GAME_STORAGE_INITIAL 2
00112
00117 #define GAME_CLEARSCORE      10000
00118
00125 #define GAME_CLEARMULTI      50
00126
00127 /*****************************/
00128 /*+ Test mode parameters +*/
00129 /*****************************/
00130
00135 #define TEST_WIDTH           8
00136
00141 #define TEST_HEIGHT          8
00142
00146 #define TEST_PUZZLE          false
00147
00153 #define TEST_LINES           true
00154
00159 #define TEST_NODELAY         true
00160
00165 #define TEST_EASY            false
00166
00172 #define TEST_WRAP            false
00173
00178 #define TEST_BOMBS           false
00179
00184 #define TEST_UPGRADES        true
00185
00191 #define TEST_REFILL_MODE     1
00192
00197 #define TEST_STORAGE_INITIAL 2
00198
00203 #define TEST_BALANCED        true
00204
00205 /************************/
00206 /*+ Solver limitation +*/
00207 /************************/
00208
00214
00215 #define SOLVER_MAX_CONFIGS                                          \
00216     4000000
00218
00219 #endif // _PROJECT_PARAMETERS_H_
```

4.19 Référence du fichier project_tools.h

Tools.

```
#include <SDL2/SDL.h>
#include <SDL2/SDL_timer.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
```

Macros

— #define MAX(a, b)

Macro calculant le maximum de deux valeurs. Ne pas utiliser avec des expressions faisant des effets de bord ! Par exemple, MAX(x++, y) risque d'incrémenter la variable x deux fois (car si elle est plus grande que y, x++ est évalué dans (a) > (b), puis dans (a)).

— #define MIN(a, b)

Macro calculant le minimum de deux valeurs. Ne pas utiliser avec des expressions faisant des effets de bord ! Par exemple, MIN(x++, y) risque d'incrémenter la variable x deux fois (car si elle est plus grande que y, x++ est évalué dans (a) < (b), puis dans (a)).

— #define SWAP(a, b)

Macro pour échanger deux valeurs.

Définitions de type

— typedef unsigned char uchar

— typedef unsigned short ushort

— typedef unsigned int uint

— typedef unsigned long ulong

4.19.1 Description détaillée

Tools.

Quelques macros et définitions de types que vous pouvez utiliser dans votre projet

Remarques

> Vous ne devez PAS modifier ce fichier car les macros sont utilisés dans la mise en oeuvre de l'interface graphique.

Définition dans le fichier project_tools.h.

4.19.2 Documentation des macros

4.19.2.1 MAX

```
#define MAX(
            a,
            b)
```

Valeur :

```
((a) > (b) ? (a) : (b))
```

Macro calculant le maximum de deux valeurs. Ne pas utiliser avec des expressions faisant des effets de bord ! Par exemple, MAX(x++, y) risque d'incrémenter la variable x deux fois (car si elle est plus grande que y, x++ est évalué dans (a) > (b), puis dans (a)).

Définition à la ligne 44 du fichier project_tools.h.

4.19.2.2 MIN

```
#define MIN(
            a,
            b)
```

Valeur :

```
((a) < (b) ? (a) : (b))
```

Macro calculant le minimum de deux valeurs. Ne pas utiliser avec des expressions faisant des effets de bord ! Par exemple, MIN(x++, y) risque d'incrémenter la variable x deux fois (car si elle est plus grande que y, x++ est évalué dans (a) < (b), puis dans (a)).

Définition à la ligne 53 du fichier project_tools.h.

4.19.2.3 SWAP

```
#define SWAP(
            a,
            b)
```

Valeur :

```
    do {                                                    \
        typeof(a) temp = a;                                 \
        a              = b;                                 \
        b              = temp;                              \
    } while (0)
```

Macro pour échanger deux valeurs.

Définition à la ligne 59 du fichier project_tools.h.

4.19.3 Documentation des définitions de type

4.19.3.1 uchar

```
typedef unsigned char uchar
```

Définition à la ligne 32 du fichier project_tools.h.

4.19.3.2 ushort

```
typedef unsigned short ushort
```

Définition à la ligne 33 du fichier project_tools.h.

4.19.3.3 uint

```
typedef unsigned int uint
```

Définition à la ligne 34 du fichier project_tools.h.

4.19.3.4 ulong

```
typedef unsigned long ulong
```

Définition à la ligne 35 du fichier project_tools.h.

4.20 project_tools.h

Aller à la documentation de ce fichier.

```
00001
00015
00016 #ifndef _TOOLS_H_
00017 #define _TOOLS_H_
00018
00019 #include <SDL2/SDL.h>
00020 #include <SDL2/SDL_timer.h>
00021 #include <stdbool.h>
00022 #include <stdio.h>
00023 #include <stdlib.h>
00024
00025 /*  _____           _      */
00026 /* |_   _|__   ___ | |___  */
00027 /*   | |/ _ \ / _ \| / __| */
00028 /*   | | (_) | (_) | \__ \ */
```

```
00029 /*   |_|\___/ \___/|_|___/ */
00030
00031 // Shortcuts for the unsigned types
00032 typedef unsigned char uchar;
00033 typedef unsigned short ushort;
00034 typedef unsigned int uint;
00035 typedef unsigned long ulong;
00036
00044 #define MAX(a, b) ((a) > (b) ? (a) : (b))
00045
00053 #define MIN(a, b) ((a) < (b) ? (a) : (b))
00054
00059 #define SWAP(a, b)                                                    \
00060     do {                                                              \
00061         typeof(a) temp = a;                                           \
00062         a              = b;                                           \
00063         b              = temp;                                        \
00064     } while (0)
00065
00066 #endif
```

Index

alloc.h, 21
    CHECK_NULL, 25
    check_null, 26
    CREATE_MEMORY, 22
    CREATE_ZERO_MEMORY, 23
    multiple_free, 26
    REDIMENSION_MEMORY, 24
AMBER
    game.h, 40
AMETHYST
    game.h, 40
ANIM_HINT
    game_play.h, 60
ANIM_IDLE
    game_play.h, 59
ANIM_KILL
    game_play.h, 60
ANIM_MOVE_BADSWAP
    game_play.h, 60
ANIM_MOVE_FALL
    game_play.h, 60
ANIM_MOVE_GOODSWAP
    game_play.h, 60
ANIM_MOVE_ROLLBACK
    game_play.h, 60
ANIM_SIZE
    game_play.h, 60
anim_type
    game_play.h, 59
ATTENTION
    error.h, 30

b
    game, 9
board, 5
    dummy, 6
board_get_cell
    game.h, 44
board_get_gravity
    game.h, 43
board_get_height
    game.h, 43
board_get_picked
    game.h, 43
board_get_width
    game.h, 42
board_read_refill
    game.h, 44
board_refill_size
    game.h, 44
BOMB
    game.h, 41
bombs
    game, 10

cell, 6
    countdown, 7
    object, 7
    power, 7
CHECK_NULL
    alloc.h, 25
check_null
    alloc.h, 26
countdown
    cell, 7
create_hash_table
    game_solver.h, 81
CREATE_MEMORY
    alloc.h, 22
CREATE_ZERO_MEMORY
    alloc.h, 23
CRITICAL
    error.h, 35
CRITICAL_L
    error.h, 36

DEBUG
    error.h, 33, 35
DEBUG_L
    error.h, 36
DEBUG_LEVEL
    error.h, 31
debug_level
    error.h, 35
delete_hash_table
    game_solver.h, 81
delete_score_table
    game_scores.h, 77

DIAMOND
    game.h, 40
dir
    solver_move, 18
direction
    game.h, 40
DOWN
    game.h, 40
dummy
    board, 6

easy
    game, 10
ELEM_SIZE
    game.h, 41
element
    game.h, 40
element_size
    hash_table, 13
EMERALD
    game.h, 40
EMPTY
    game.h, 41
end_signal
    game_play.h, 60
ERROR
    error.h, 34, 35
error.h, 28
    ATTENTION, 30
    CRITICAL, 35
    CRITICAL_L, 36
    DEBUG, 33, 35
    DEBUG_L, 36
    DEBUG_LEVEL, 31
    debug_level, 35
    ERROR, 34, 35
    ERROR_L, 36
    FATAL, 32
    HINT, 30
    INFO, 34, 35
    INFO_L, 36
    KO, 31
    msg_level, 35
    OFF, 35
    OK, 30
    PRINT_DEBUG, 33
    PRINT_ERROR, 31
    TRACE, 33, 35
    TRACE_L, 36
    WARNING, 34, 35
    WARNING_L, 36
ERROR_L
    error.h, 36

FATAL
    error.h, 32

game, 7
    b, 9
    bombs, 10
    easy, 10
    gravity_control, 10
    gravity_gems, 10
    lines, 9
    nodelay, 10
    puzzle, 9
    refill_mode, 11
    score, 11
    upgrades, 9
game.h, 37
    AMBER, 40
    AMETHYST, 40
    board_get_cell, 44
    board_get_gravity, 43
    board_get_height, 43
    board_get_picked, 43
    board_get_width, 42
    board_read_refill, 44
    board_refill_size, 44
    BOMB, 41
    DIAMOND, 40
    direction, 40
    DOWN, 40
    ELEM_SIZE, 41
    element, 40
    EMERALD, 40
    EMPTY, 41
    game_clone, 48
    game_create_classic, 45
    game_create_puzzle, 46
    game_next_level, 48
    game_release, 48
    GEM_BASIC, 41
    GEM_FLAME, 41
    GEM_GRAVITY, 41
    gem_power, 41
    GEM_SIZE, 41
    GEM_STAR, 41
    HYPERCUBE, 41
    LEFT, 40
    RIGHT, 40
    ROCK, 41
    RUBY, 40
    SAPPHIRE, 40

SC_CONSUMED, 42
SC_FLAMED, 42
SC_GLOBAL, 42
SC_GRAVITY, 42
SC_HYPERCUBES, 42
SC_LEVEL, 42
SC_SIZE, 42
SC_STARRED, 42
SC_SWAPS, 42
SC_TIME, 42
score_type, 41
SIZE_DIRECTION, 40
TOPAZ, 41
UP, 40
game_anim_signal
game_play.h, 68
GAME_BALANCED
project_parameters.h, 89
GAME_BOMBS
project_parameters.h, 89
game_classic_end
game_play.h, 62
game_classic_score
game_play.h, 61
GAME_CLEARMULTI
project_parameters.h, 90
GAME_CLEARSCORE
project_parameters.h, 90
game_clone
game.h, 48
game_create_classic
game.h, 45
game_create_puzzle
game.h, 46
game_cycle_gravity
game_play.h, 64
GAME_EASY
project_parameters.h, 88
game_end_animation
game_play.h, 65
game_end_signal
game_play.h, 68
game_get_cell_kill
game_play.h, 67
game_get_cell_move
game_play.h, 66
game_handle_detonation
game_play.h, 64
game_handle_hint_request
game_play.h, 63
game_handle_mouse_click
game_play.h, 62
game_handle_refill
game_play.h, 65
GAME_HEIGHT
project_parameters.h, 88
game_ias.h, 53
IA_AGRESSIVE, 54
ia_agressive, 55
ia_classic, 54
IA_CLASSIC_SIZE, 54
ia_functions, 56
ia_names, 56
IA_RANDOM, 54
ia_random, 54
IA_SAFE, 54
ia_safe, 54
game_implement_solver_move
game_solver.h, 83
game_init_animation
game_play.h, 61
GAME_LINES
project_parameters.h, 88
game_next_level
game.h, 48
GAME_NODELAY
project_parameters.h, 88
game_play.h, 57
ANIM_HINT, 60
ANIM_IDLE, 59
ANIM_KILL, 60
ANIM_MOVE_BADSWAP, 60
ANIM_MOVE_FALL, 60
ANIM_MOVE_GOODSWAP, 60
ANIM_MOVE_ROLLBACK, 60
ANIM_SIZE, 60
anim_type, 59
end_signal, 60
game_anim_signal, 68
game_classic_end, 62
game_classic_score, 61
game_cycle_gravity, 64
game_end_animation, 65
game_end_signal, 68
game_get_cell_kill, 67
game_get_cell_move, 66
game_handle_detonation, 64
game_handle_hint_request, 63
game_handle_mouse_click, 62
game_handle_refill, 65
game_init_animation, 61
game_puzzle_end, 62

game_retrieve_hint, 67
game_set_gravity, 64
gem_kill, 60
KILL_EXPLODE, 61
KILL_EXPLODE_RING, 61
KILL_NOKILL, 61
KILL_SHATTER, 61
KILL_SHRINK, 61
KILL_SIZE, 61
SIGNAL_END_CLGAME, 60
SIGNAL_END_LEVEL, 60
SIGNAL_END_PUZZLE, 60
SIGNAL_SIZE, 60
game_puzzle_end
game_play.h, 62
GAME_REFILL_MODE
project_parameters.h, 89
game_release
game.h, 48
game_retrieve_hint
game_play.h, 67
game_save.h, 71
load_game, 73
save_game, 73
save_keyword_strings, 74
save_keywords, 72
SK_BOARD, 72
SK_BOMBS, 73
SK_EASY, 72
SK_GRAVITY, 72
SK_GRAVITY_CONTROL, 73
SK_HEIGHT, 72
SK_LEVEL, 73
SK_LEVEL_SCORE, 73
SK_LINES, 72
SK_NODELAY, 72
SK_PUZZLE, 72
SK_REFILL, 73
SK_REFILL_COUNT, 73
SK_REFILL_QUEUE, 72
SK_SCALE_SCORE, 73
SK_SCORE_ARRAY, 73
SK_SIZE, 73
SK_TARGET_SCORE, 73
SK_UPGRADES, 72
SK_WIDTH, 72
SK_WRAP, 73
game_scores.h, 75
delete_score_table, 77
load_score_table, 76
save_score_table, 76
sort_score_table, 77
game_set_gravity
game_play.h, 64
game_solver.h, 78
create_hash_table, 81
delete_hash_table, 81
game_implement_solver_move, 83
hash_table_insert, 82
solve_puzzle, 82
solver, 81
SOLVER_ASTAR, 81
SOLVER_BFS, 81
SOLVER_CYCLE, 80
SOLVER_DFS, 81
SOLVER_GRAVITY, 80
solver_move, 80
solver_move_type, 80
solver_result_type, 80
SOLVER_SOLVED, 81
SOLVER_STORAGE, 80
SOLVER_SWAP, 80
SOLVER_TIMEOUT, 80
SOLVER_UNSOLVABLE, 80
GAME_STORAGE_INITIAL
project_parameters.h, 90
GAME_UPGRADES
project_parameters.h, 89
GAME_WIDTH
project_parameters.h, 88
GAME_WRAP
project_parameters.h, 89
GEM_BASIC
game.h, 41
GEM_FLAME
game.h, 41
GEM_GRAVITY
game.h, 41
gem_kill
game_play.h, 60
gem_power
game.h, 41
GEM_SIZE
game.h, 41
GEM_STAR
game.h, 41
gravity_control
game, 10
gravity_gems
game, 10

hash_capacity
hash_table, 13

hash_size
  hash_table, 13
hash_table, 11
  element_size, 13
  hash_capacity, 13
  hash_size, 13
  hash_table, 13
hash_table_insert
  game_solver.h, 82
HINT
  error.h, 30
HYPERCUBE
  game.h, 41

IA_AGRESSIVE
  game_ias.h, 54
ia_agressive
  game_ias.h, 55
ia_classic
  game_ias.h, 54
IA_CLASSIC_SIZE
  game_ias.h, 54
ia_functions
  game_ias.h, 56
ia_names
  game_ias.h, 56
IA_RANDOM
  game_ias.h, 54
ia_random
  game_ias.h, 54
IA_SAFE
  game_ias.h, 54
ia_safe
  game_ias.h, 54
index
  solver_move, 18
INFO
  error.h, 34, 35
INFO_L
  error.h, 36

KILL_EXPLODE
  game_play.h, 61
KILL_EXPLODE_RING
  game_play.h, 61
KILL_NOKILL
  game_play.h, 61
KILL_SHATTER
  game_play.h, 61
KILL_SHRINK
  game_play.h, 61
KILL_SIZE
  game_play.h, 61
KO
  error.h, 31

LEFT
  game.h, 40
level
  score_block, 14
level_score
  score_block, 14
lines
  game, 9
load_game
  game_save.h, 73
load_score_table
  game_scores.h, 76

MAX
  project_tools.h, 96
MIN
  project_tools.h, 96
msg_level
  error.h, 35
multiple_free
  alloc.h, 26

names
  score_table, 16
next
  solver_move, 19
nodelay
  game, 10

object
  cell, 7
OFF
  error.h, 35
OK
  error.h, 30

permutation
  score_table, 17
player_index
  score_table, 17
power
  cell, 7
PRINT_DEBUG
  error.h, 33
PRINT_ERROR
  error.h, 31
project_parameters.h, 85
  GAME_BALANCED, 89
  GAME_BOMBS, 89

GAME_CLEARMULTI, 90
GAME_CLEARSCORE, 90
GAME_EASY, 88
GAME_HEIGHT, 88
GAME_LINES, 88
GAME_NODELAY, 88
GAME_REFILL_MODE, 89
GAME_STORAGE_INITIAL, 90
GAME_UPGRADES, 89
GAME_WIDTH, 88
GAME_WRAP, 89
SOLVER_MAX_CONFIGS, 93
TEST_BALANCED, 92
TEST_BOMBS, 92
TEST_EASY, 91
TEST_HEIGHT, 90
TEST_LINES, 91
TEST_NODELAY, 91
TEST_PUZZLE, 91
TEST_REFILL_MODE, 92
TEST_STORAGE_INITIAL, 92
TEST_UPGRADES, 92
TEST_WIDTH, 90
TEST_WRAP, 91
WINDOW_SCREENHEIGHT, 87
WINDOW_SCREENWIDTH, 87
project_tools.h, 95
MAX, 96
MIN, 96
SWAP, 96
uchar, 97
uint, 97
ulong, 97
ushort, 97
puzzle
game, 9

REDIMENSION_MEMORY
alloc.h, 24
refill_mode
game, 11
refills
score_block, 15
RIGHT
game.h, 40
ROCK
game.h, 41
RUBY
game.h, 40

SAPPHIRE
game.h, 40
save_game
game_save.h, 73
save_keyword_strings
game_save.h, 74
save_keywords
game_save.h, 72
save_score_table
game_scores.h, 76
SC_CONSUMED
game.h, 42
SC_FLAMED
game.h, 42
SC_GLOBAL
game.h, 42
SC_GRAVITY
game.h, 42
SC_HYPERCUBES
game.h, 42
SC_LEVEL
game.h, 42
SC_SIZE
game.h, 42
SC_STARRED
game.h, 42
SC_SWAPS
game.h, 42
SC_TIME
game.h, 42
scale_score
score_block, 15
score
game, 11
score_block, 13
level, 14
level_score, 14
refills, 15
scale_score, 15
scores, 15
target_score, 14
score_table, 15
names, 16
permutation, 17
player_index, 17
scores, 16
size, 17
score_type
game.h, 41
scores
score_block, 15
score_table, 16
SIGNAL_END_CLGAME

game_play.h, 60
SIGNAL_END_LEVEL
game_play.h, 60
SIGNAL_END_PUZZLE
game_play.h, 60
SIGNAL_SIZE
game_play.h, 60
size
score_table, 17
SIZE_DIRECTION
game.h, 40
SK_BOARD
game_save.h, 72
SK_BOMBS
game_save.h, 73
SK_EASY
game_save.h, 72
SK_GRAVITY
game_save.h, 72
SK_GRAVITY_CONTROL
game_save.h, 73
SK_HEIGHT
game_save.h, 72
SK_LEVEL
game_save.h, 73
SK_LEVEL_SCORE
game_save.h, 73
SK_LINES
game_save.h, 72
SK_NODELAY
game_save.h, 72
SK_PUZZLE
game_save.h, 72
SK_REFILL
game_save.h, 73
SK_REFILL_COUNT
game_save.h, 73
SK_REFILL_QUEUE
game_save.h, 72
SK_SCALE_SCORE
game_save.h, 73
SK_SCORE_ARRAY
game_save.h, 73
SK_SIZE
game_save.h, 73
SK_TARGET_SCORE
game_save.h, 73
SK_UPGRADES
game_save.h, 72
SK_WIDTH
game_save.h, 72
SK_WRAP
game_save.h, 73
solve_puzzle
game_solver.h, 82
solver
game_solver.h, 81
SOLVER_ASTAR
game_solver.h, 81
SOLVER_BFS
game_solver.h, 81
SOLVER_CYCLE
game_solver.h, 80
SOLVER_DFS
game_solver.h, 81
SOLVER_GRAVITY
game_solver.h, 80
SOLVER_MAX_CONFIGS
project_parameters.h, 93
solver_move, 17
dir, 18
game_solver.h, 80
index, 18
next, 19
type, 18
solver_move_type
game_solver.h, 80
solver_result_type
game_solver.h, 80
SOLVER_SOLVED
game_solver.h, 81
SOLVER_STORAGE
game_solver.h, 80
SOLVER_SWAP
game_solver.h, 80
SOLVER_TIMEOUT
game_solver.h, 80
SOLVER_UNSOLVABLE
game_solver.h, 80
sort_score_table
game_scores.h, 77
SWAP
project_tools.h, 96

target_score
score_block, 14
TEST_BALANCED
project_parameters.h, 92
TEST_BOMBS
project_parameters.h, 92
TEST_EASY
project_parameters.h, 91
TEST_HEIGHT

project_parameters.h, 90
TEST_LINES
project_parameters.h, 91
TEST_NODELAY
project_parameters.h, 91
TEST_PUZZLE
project_parameters.h, 91
TEST_REFILL_MODE
project_parameters.h, 92
TEST_STORAGE_INITIAL
project_parameters.h, 92
TEST_UPGRADES
project_parameters.h, 92
TEST_WIDTH
project_parameters.h, 90
TEST_WRAP
project_parameters.h, 91
TOPAZ
game.h, 41
TRACE
error.h, 33, 35
TRACE_L
error.h, 36
type
solver_move, 18

uchar
project_tools.h, 97
uint
project_tools.h, 97
ulong
project_tools.h, 97
UP
game.h, 40
upgrades
game, 9
ushort
project_tools.h, 97

WARNING
error.h, 34, 35
WARNING_L
error.h, 36
WINDOW_SCREENHEIGHT
project_parameters.h, 87
WINDOW_SCREENWIDTH
project_parameters.h, 87